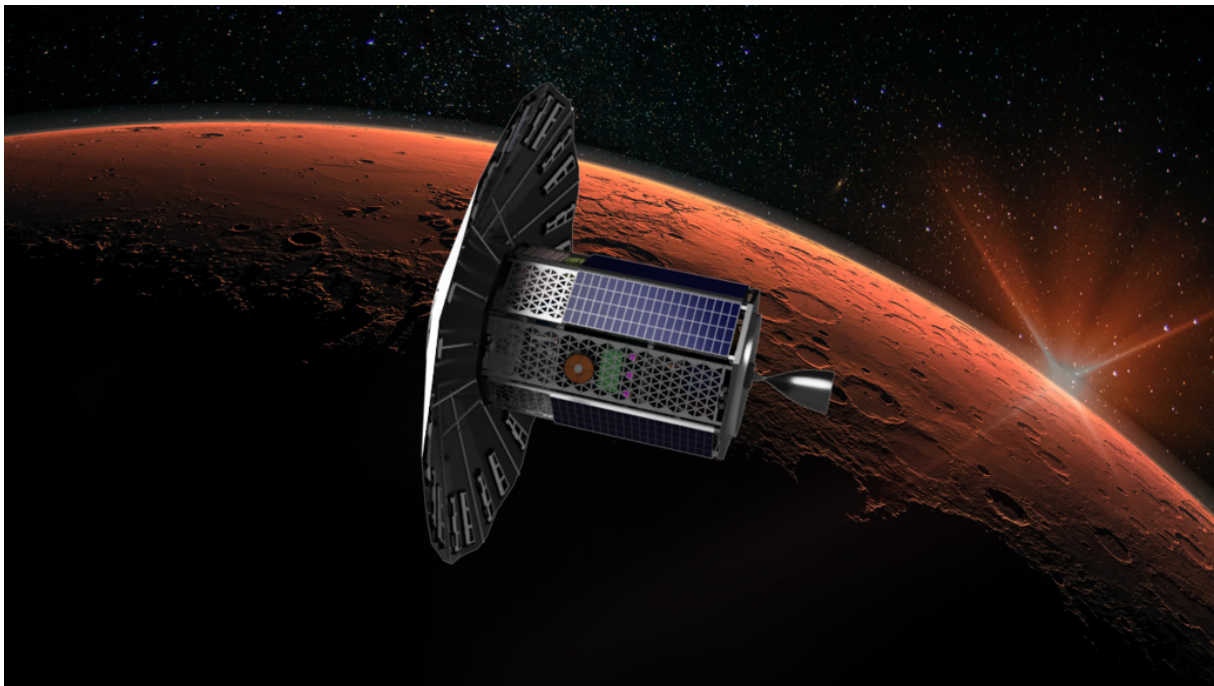


Aerocapture Manoeuvre: Analysis of Atmospheric and Space Flight Dynamics and Trajectory Optimization

Aerothermal design ADM06



Student: Segundo Urreza Atue

CID: 01727798

Date: 20/06/2022

Academic responsible: Dr Paul Bruce

Supervisor: Dr Paul Bruce

Department: Department of Aeronautics

Course: H401/H415 MEng

Aeronautical Engineering

Module: AERO60004 - Group Design
Project

Academic year: 2021/2022

Department of Aeronautics
South Kensington Campus
Imperial College London
London SW7 2AZ
U.K.



Abstract

The atmospheric and space flight dynamics of a Martian aerocapture demonstrator spacecraft were analyzed in order to optimise a trans-atmospheric trajectory and adequately select the design point, consisting of the ballistic coefficient (β), flight path angle (γ) at atmospheric interface (AI) and the design Lift-to-Drag ratio (L/D). The re-entry dynamics of the spacecraft were simulated using a high fidelity three-degree-of-freedom motion model which includes Martian specific dynamics, like it's rotational period, as well as coriolis accelerations imparted by the rotation of the atmosphere. Firstly, an aerocapture corridor was constructed using the *D-SAT algorithm*. This algorithm consist of "trial-and-error" trajectory simulations, iterating over a range of β and allocating a range of γ that will yield a successful aerocapture manoeuvre. In addition to trajectory modelling, the *D-SAT algorithm* will analyse the aerothermal environment experienced during re-entry using thermal models based on *Sutton-Graves*, *Tauber-Sutton* and *Stephan-Boltzmann* relations. Subsequently, the *AsTRO algorithm* will select an optimal trajectory from a large range of possible trajectories by performing an optimisation process. The optimisation process will consider spacecraft geometric and trajectory features like: volume of deployable structure, propellant mass required for post-aerocapture orbit corrective manoeuvres, trajectory thermal properties like thermal load (Q), maximum heat flux $\dot{q}_{max}$ and maximum aeroshell wall temperature (T_W) and final post-aerocapture orbital period ($T_{orbital}$). This optimisation will yield the design point which will critically influence the geometry of the spacecraft as well as the design of the inter-planetary orbit. Both the *D-SAT algorithm* and *AsTRO algorithm* have been developed by the author of this report specifically for this project and can easily be extrapolated to other Aerocapture manoeuvres.

Sub-Group	Name	Role
ADM01: Project Coordination	Camille Boutrolle Jia Hui Sofia Tan Jinlun Jim Zhang	Project Coordinator
ADM02: Mission Design and Analysis	Aaditi Vaze Luca Mazzotta Mohit Tank Yago Molano Gomez	Interplanetary Trajectory Design & Optimisation Interplanetary Trajectory Design & Optimisation Mission Architecture & Future Roadmap Hyperbolic Trajectory Design & Modelling
ADM03: Spacecraft Systems & Control	Chorley King Dannielle Lee Lucas Parkins Nihal Simha	Command & Data Handling & Power Systems Guidance & Navigation Telemetry Tracking & Command Attitude Determination & Control
ADM04: Spacecraft Static Structural Design	Samuel Cheung Sze Hang Prudence Lee	Static Structural Design & CAD Static Structural Design & CAD
ADM05: Spacecraft Dynamic Structural Design	Julia Wiszniowska Luc Groshens Luke Wyatt	Deployable Panel System Structural Design Deployable Rib & Spar System Structural Design Deployable Mechanism Structural Design & TPS
ADM06: Spacecraft Aerothermal Design	Claire Namuroy Erikas Kymantas Jaime Galiana Herrera Segundo Urraza Atue	Heat Transfer & Pressure Modelling & CFD Atmospheric & Space Flight Dynamics Hypersonic Stability Analysis Atmospheric & Space Flight Dynamics
ADM07: Spacecraft & Environment Interaction	Jae Min Oh Livia Trambaiolo Mohammed Choudhury Yasmine Chan	Radiation & Debris Protection Instrumentation & Scientific Payload Thermal Management Emergency Planning



Contents

List of Figures	iv
List of Tables	v
1 Introduction	1
2 Atmospheric and Space Flight Dynamics	1
2.1 Atmospheric models	2
2.1.1 Perifocal Gravity Turn model	2
2.1.2 Three Dimensional Re-entry Dynamics (6 States)	2
2.2 Space flight model	3
3 Aerothermal model	3
3.1 Aerodynamics of Re-entry	3
3.2 Thermal model	3
4 Sensitivity Analysis of Dynamic Models	4
4.1 Initial conditions	4
4.2 Determination of Successful Aerocapture Trajectories (D-SAT) algorithm	5
4.3 Sensitivity Analysis of Dynamics	6
4.3.1 Lift-to-Drag Ratio (L/D)	6
4.4 Drag Coefficient (C_D)	7
4.5 Scattering of Atmospheric Properties	7
5 Aerocapture (successful) Trajectory Robust Optimisation (AsTRO) algorithm	8
5.1 Constraint Conditions	8
5.2 Performance drivers (Features)	8
5.2.1 Volume of deployable structure ($V_{structural}$)	8
5.2.2 ΔV budget for orbital correction and Orbital period ($T_{orbital}$)	8
5.2.3 Aerothermal environment (Q , $\dot{q}_{max}$ and T_W)	9
5.3 Feature Scaling: Standarization	9
5.4 Importance of each feature	9
5.5 Loss function architecture	10
5.5.1 Least-Square method	10
5.5.2 Pseudo-Huber loss function	10
5.5.3 Loss function comparison	10
5.6 AsTRO Algorithm Sensitivity Analysis and Results	11



6 Design point	12
7 Future Mission Design	12
Appendices	14
A Reference frames used in modelling dynamics	14
B Augmented three-Dimensional Reentry Dynamics (13 states)	14
C Orbital Element Determination Algorithm	15
D Thermal model constants for Martian atmosphere	16
E Aerocapture sensitivity figures	17
E.1 Comparison between Ballistic and Lifted re-entries	17
E.2 Atmospheric model variations	17
F Derivation of performance drivers used in cost function	18
F.1 Volume of dynamical structure	18
F.2 Post-aerocapture orbital correction manoeuvres	19
G MATLAB Code: D-SAT Algorithm	19
G.1 MATLAB Code: Trajectory simulations	21
G.2 MATLAB Code: Re-entry dynamics	26
H MATLAB Code: AsTRO Algorithm	27
H.1 MATLAB Code: Cost Function	30



List of Figures

1	Aerocapture trajectory depiction [5]	1
2	Drag coefficient (C_D) vs Mach number (M) for an arbitrary body: As suggested, C_D has an asymptotic behaviour as M increases[1]	4
3	Variations in Knudsen number (Kn) along atmospheric pass	4
4	Aerocapture corridor: This corridor corresponds to a lifted re-entry ($L/D = 0.2$) with a $\pm 3\sigma$ atmospheric parameters uncertainty	5
5	Drag uncertainty robust aerocapture corridor:	7
6	Loss function comparison	11
7	Noise robustness behaviour of loss functions	11
8	Perifocal gravity turn model frame of reference: Perifocal coordinate (PQW) system	14
9	Three Dimensional Reentry Dynamics frame of reference: Planet fixed and local horizon frames	14
10	Mars specific function ($f_i(V)$) for radiative heat flux modelling	16
11	Aerocapture corridor sensitivity to Lift-to-Drag ratio and comparison of corridor width	17
12	Ballistic Re-entry corridor: Sensitivity to scattering of atmospheric properties	17
13	Lifted Re-entry corridor: Sensitivity to scattering of atmospheric properties	18
14	Retracted and deployed visualization of deployable structure	18
15	Periapsis raise and Circularization manoeuvre	19



List of Tables

1	Mission specifications from ESAs statement of work [4]	6
2	Weight vector (k) influence on optima	11
3	Atmospheric model influence on optima	11
4	Atmospheric interface location influence on optima	11
5	Final design parameters, trajectory properties and post-aerocapture orbital elements	12



Nomenclature

Symbols

A	Cross-sectional area of heat shield (m^2)
r	Radius of heat shield (m)
L/D	Lift-to-Drag ratio
C_D	Drag coefficient
Q	Heat load (J)
T_W	Thermal protective system wall temperature (K)
$\dot{q}$	Heat flux (W m^{-2})
$\dot{q}_{max}$	Maximum heat flux (W m^{-2})
$T_{orbital}$	Orbital period of orbit (s)
m	Mass of the spacecraft (Kg)
V	Velocity of spacecraft (m s^{-1}) or Volume (m^3)
R	Position of spacecraft from celestial body's center (m)
D	Drag experience by spacecraft (N)
L	Lift experience by spacecraft (N)
X, Y, Z	Wind axis, where X is parallel to velocity vector, Z is parallel to zenith and Y complete coordinate system
F_X, F_Y, F_Z	Forces acting in wind axis (N)
ΔV	Change in orbital velocity (m s^{-1})
μ	Standard gravitational parameter of celestial body ($\text{m}^3 \text{s}^{-2}$)
ν	True anomaly in perifocal frame of reference ($^\circ$) or Kinematic viscosity ($\text{m}^2 \text{s}^{-1}$)
γ	Flight path angle ($^\circ$)
$\Delta\gamma$	Width of aerocapture corridor ($^\circ$)
β	Ballistic coefficient (Kg m^{-2})
δ	Longitude, positive towards east ($^\circ$)
λ	Latitude, positive towards North ($^\circ$)
χ	Heading angle ($90^\circ - A$), positive from East to North ($^\circ$)
Ω	Rotational rate of planet (rad s^{-1}) or Argument of right ascending node (rad)
i	Inclination of orbit (rad)
e	Eccentricity of orbit (rad)
a	Semi-major axis of orbit (m)
α	Angle of attach (rad)
σ	Standard deviation
Kn	Knudsen number
c	Molecular mean velocity (m s^{-1})
$\bar{R}$	Gas constant (8.3144598) ($\text{J mol}^{-1} \text{K}^{-1}$)
$\mathbf{x}$	State vector or feature vector, depends on subscript
k	Weight vector for each feature (optimisation)

Abbreviations

TPS	Thermal protective system
PQW	Perifocal coordinate system
AI	Atmospheric interface
AE	Atmospheric exit
ODE	Ordinary Differential Equation AoA
Angle of Attack	
$DMCS$	Direct Monte Carlo Simualtions

1 Introduction

Over the last decades, aeroassisted trajectories have stopped being a work of fiction and have become an important field of research in the aerospace industry. Before examining the dynamics of these trajectories, a simple question must be asked. What is an Aerocapture manoeuvre? To answer this question we must address the basics of aeroassisted manoeuvres, starting with aerobraking. This manoeuvre is one of those hypothesised trajectories, which was first executed by the Magellan spacecraft in 1992, laying the groundwork for future missions. Aerobraking is used to reduce the apoapsis of an elliptical orbit around a celestial body with an atmosphere. It does so by passing through the higher reaches of the atmosphere, where the resulting drag will decrease the orbital speed, which reduces the semi-major axis of the orbit. Despite the concept having been proved in 1992, it was no sooner than 1997 when the Mars Global Surveyor (MGS) became the first spacecraft to use this manoeuvre as an integral part of an inter-planetary mission.

On the other hand, aerocapture, despite using the same physical principal, is not identical to aerobraking. When an interplanetary spacecraft arrives at its destination, it must perform an insertion burn in order to migrate from a hyperbolic orbit to an elliptical orbit. This manoeuvre requires a considerable cut-down in orbital velocity, usually in the order of kilometers per second. From the rocket equation, a change in velocity (ΔV) of this magnitude requires a large portion of the spacecraft's mass to be allocated to propellant, reducing the scientific payload of the mission. So what is aerocapture? Given that the targeted body possesses a sufficiently dense atmosphere, a spacecraft can perform a deep atmospheric dive which would be enough to provide changes in velocities of the order of kilometers per second, enough to modify the nature of the orbit, gravitationally binding the spacecraft to the celestial body.

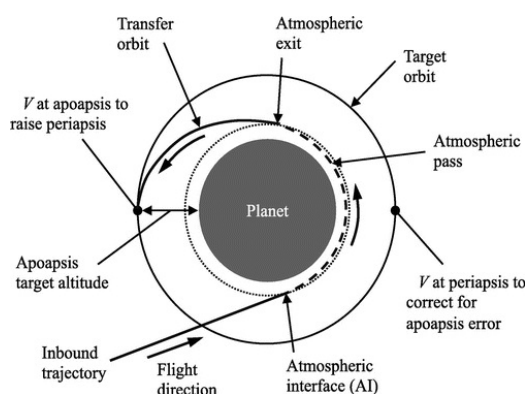


Figure 1: Aerocapture trajectory depiction [5]

Because of this situation, accurate modelling of the atmospheric and space flight dynamics, as well as a detailed atmospheric and aerothermal models, is essential in order to determine the attitude and kinematic requirements for a successful aerocapture trajectory. Due to the nature of a deep atmospheric dive, a series of problems arise, which include: unpredictable turbulence effects, variations in atmospheric properties, limitations in insertion accuracy, hypersonic instabilities and most importantly unprecedented thermal loads. All these complications must be taken into account to guarantee the success of the mission.

The ambition of this report is to analyse the dynamics of atmospheric and space flight in depth in order to construct two separate algorithms. The first one, named *Determination of Successful Aerocapture Trajectories (D-SAT) algorithm*, attempts to determine all possible successful aerocapture trajectories while the second algorithm, *Aerocapture (successful) Trajectory Robust Optimization (AsTRO) algorithm*, will analyse the results outputted by the D-SAT algorithm and find the optimal design parameters and trajectory for the mission. The optimization will select the most beneficial trajectory taking into consideration mission and spacecraft design drivers, such as volume of deployable structure ($V_{structural}$), ΔV for post-aerocapture orbital corrections, heat loads (Q), maximum heat flux ($\dot{q}_{max}$), aeroshell wall temperature (T_W) and post-aerocapture orbital period ($T_{orbital}$).

2 Atmospheric and Space Flight Dynamics

This section will introduce the two models used to simulate the atmospheric and exo-atmospheric trajectories of the spacecraft, these are the *Perifocal gravity turn model* and the *Three Dimensional Re-entry Dynamics model*. The selected models are superior to other approaches due to the fact that they can be used to model trans-atmospheric as well as exo-atmospheric trajectories of the spacecraft by simply neglecting aerothermal effects imparted by the atmosphere. A shared aspect of these models is that they idealize the spacecraft as a point mass and both use a planet-centered, non rotating reference frame as well as being represented in a state space format. The main difference between the models is that the initial approach only manages to capture the dynamics in two dimensions, where as the succeeding model is able to capture all three dimensions.



2.1 Atmospheric models

2.1.1 Perifocal Gravity Turn model

This model uses four states to simulate the trajectory of a point mass under the gravitational effects of a body as well as the aerodynamics effect imparted by the planet's atmosphere. This model uses a perifocal coordinate (PQW) system where the origin is located at the focus of the orbit, which coincides with the celestial body. The state equations are the following[7]:

$$\dot{V} = -\frac{\mu \sin(\gamma)}{R^2} - \frac{D}{m} \quad (1)$$

$$\dot{\gamma} = \frac{\mu \cos(\gamma)}{VR^2} + \frac{L}{Vm} + \frac{V \cos(\gamma)}{R} \quad (2)$$

$$\dot{\nu} = \frac{V \cos(\gamma)}{R} \quad (3)$$

$$\dot{R} = \dot{h} = V \sin(\gamma) \quad (4)$$

Where V is velocity, μ is the gravitational parameter, γ is the flight path angle (FPA), R is the radius from the center of the orbiting body, D is aerodynamic drag, m is the spacecraft's mass, L is aerodynamics lift, ν is the true anomaly from atmospheric interface (AI) and h is altitude from the orbiting body's surface.

The most beneficial property of this model is its simplicity, a fourth order state space model can be solved by a simple ODE solver like a non-stiff explicit Runge-Kutta scheme, reducing the computation time and allowing extensive number of iterations to be made. However, there are certain disadvantages to modelling the dynamics using this model. As per section 4, the success of the mission is reliant on the atmospheric properties of the planet. Not being able to accurately predict the position of the spacecraft in the planet's atmosphere will not allow an accurate determination of atmospheric parameters. This will reduce the fidelity of the results, since atmospheric properties change with location, it is analogous to modelling the temperature of the North pole with a model constructed for predicting equatorial temperatures. In addition, this model does not take into account the rotation of the planet or any Coriolis accelerations. This behaviour will be proved to be significant in section 4. Despite being a simplistic model, the perifocal dynamics were accurate enough to start the analysis of atmospheric dynamics and design process of the aerocapture trajectory and will be compared to the other model later on.

2.1.2 Three Dimensional Re-entry Dynamics (6 States)

In contrast to the previous model, this model attempts to predict the three dimensional dynamics of the spacecraft with respect to a planet-centered, non rotating frame. The dynamics are represented by a sixth order state space model, which is divided into two categories, these are kinematics and kinetics of particle motion.

The kinematic equations of particle motion are represented by the following state equations [8]:

$$\dot{\delta} = \frac{V \cos(\gamma) \cos(\chi)}{R \cos(\lambda)} - \Omega \quad , \quad \dot{\lambda} = \frac{V \cos(\gamma) \sin(\chi)}{R} \quad , \quad \dot{R} = V \sin(\gamma) \quad (5)$$

The kinetic equations of particle motion (inside the atmosphere) are represented by the following state equations [8]:

$$\dot{V} = -\frac{\mu \sin(\gamma)}{R^2} + \frac{F_X}{m} + \Omega^2 R \cos(\lambda) [\sin(\gamma) \cos(\lambda) - \cos(\gamma) \sin(\chi) \sin(\lambda)] \quad (6)$$

$$\dot{\gamma} = \left(\frac{V}{R} - \frac{\mu}{VR^2} \right) \cos(\gamma) + \frac{F_Y}{Vm} + 2\Omega \cos(\chi) \cos(\lambda) + \frac{\Omega^2 R \cos(\lambda)}{V} [\cos(\gamma) \cos(\lambda) + \sin(\gamma) \sin(\chi) \sin(\lambda)] \quad (7)$$

$$\dot{\chi} = \frac{-V}{R} \cos(\gamma) \cos(\chi) \tan(\lambda) + \frac{F_Z}{mV \cos(\gamma)} + 2\Omega [\tan(\gamma) \sin(\chi) \cos(\lambda) - \sin(\lambda)] \quad (8)$$

In contrast with the perifocal model, the dynamics of this model are highly accurate and take into consideration effects that were previously neglected, like the planets rotation. In section 5, special attention is given to the sensitivity of the trajectory dynamics to the Atmospheric Interface (AI) coordinates and velocity vector ($\mathbf{v}$). This was not the case for the perifocal gravity turn model. Modelling the three dimensional position of the spacecraft allows for an accurate atmospheric positioning, thus allowing for a more accurate atmospheric properties determination, increasing the fidelity of the model. This will be the model used to run simulation and determine the states at which the spacecraft must enter the atmosphere to successfully perform an aerocapture manoeuvre. This model was chosen over the two dimensional one due to its ability to model the 'hidden'



dynamics of the trajectory due to planetary rotation. Despite the model being a sixth order state space model, the system can be solved by using a stiff explicit Runge-Kutta scheme still within acceptable running times. The necessity of using a stiff ODE solver, in contrast to a non-stiff solver, reduces the accuracy of the solution. However, the ability of the solver to solve equation without becoming numerically unstable outweighs the loss in accuracy.

2.2 Space flight model

As previously mentioned, the models presented in the previous subsection are able to model space flight with extreme accuracy by simply neglecting aerothermal effects. Nevertheless, due to the nature of the iterative algorithm that will be used to optimize the trajectory, computational efficiency must be maximised to reduce the time taken per iteration. In order to do so a different method of calculating the post-aerocapture orbit was used, since a larger time span would be needed to model the entire atmospheric and exo-atmospheric trajectory. Instead, the geometry of the post aerocapture orbit is calculated using an *Orbital Element Determination* algorithm, where the pseudo code is available in appendix C. The method proposed by this algorithm relies on the position and velocity vector, $\mathbf{r}$ and $\mathbf{v}$ respectively, at atmospheric exit to calculate the orbital elements of the post-aerocapture orbit. These values are essential in order to perform an optimization of mission design drivers, like ΔV and $T_{orbital}$, this is explored further in section 5.

3 Aerothermal model

The models explained in section 2 rely on aerothermal properties of the re-entering vehicle in order to model the dynamics of the spacecraft accurately. The dynamics rely solely on the aerodynamics rather than the thermal behaviour of the spacecraft. However, accurately modelling the thermal behaviour of the flow around the spacecraft will be extremely beneficial at the time of designing a highly optimized, fail-proof and robust Thermal Protective System (TPS). The task of determining a high fidelity aerothermal model was delegated to the member in charge of the *Heating and dynamic pressure modelling estimates*[10], a quick interpretation and modifications of their findings follows.

Nearly all classical hypersonic flow theories address aerodynamics phenomena as a perfect gas in high-speed and at the upper bound of continuum gas regimes. Hypersonic flow is quantitatively described by the Mach number Independence principle, in which the Rankine-Hugoniot relationships tend to an asymptotic value as Mach number increases. This being said, most hypersonic flows encounter strong compressive shock which induce large changes in enthalpy which leads to non-equilibrium thermodynamics and quantum chemical-physics phenomena. The theory behind non-equilibrium hypersonic thermodynamics is a small research field in industry and accurate modelling methods have not yet been normalized in the aerospace industry.

3.1 Aerodynamics of Re-entry

With the purpose of accelerating the design process a series of simplifications and assumptions have been made. In order to simplify the hypersonic flow, the Mach Independence principle as well as continuum flow regime throughout the trajectory were assumed, allowing for more simplistic methods for aerodynamic drag determination. The drag coefficient (C_D) was assumed to be constant, from modified Newtonian theory, the calculated C_D was 1.55. From figure 2, it can be seen how this simplification is acceptable. Nevertheless, the second assumption is less accurate than the first one. From figure 3, it can be seen that a large portion atmospheric trajectory is spent in the region between Continuum and free-molecular flow, known as the slip-flow regime. Meaning that the drag calculations will not be completely accurate, this is because drag follows the relationship depicted equation 9. However, from data analysis performed with the Viking 1 aeroshell flight data [2] and upon consulting Dr. Bruce, drag coefficients in the free-molecular flow are usually 50% to 100% larger than continuum regime. In addition, C_D s in the slip-flow regime, tend to increase by 20% relative to continuum flow, for a first iteration of the thermal design, the drag will be kept constant with further analysis on drag analysis in section 4.

$$D = \frac{1}{2} \rho V^2 S_{ref} C_D (Kn, M, Re) \quad (9)$$

In addition, the lift generated by the spacecraft will be assumed to be proportional to the drag generated. The proportionality constant is known as the Lift-to-Drag (L/D) and the range [0 , 0.2] will be analyzed.

3.2 Thermal model

The thermal model used in the D-SAT algorithm models thermal properties like heat flux ($\dot{q}_{max}$), heat load (Q) and aeroshell wall temperature (T_W) in a simplistic fashion. The modelling is carried forward for a TPS design point of view and not to characterize the flow over the aeroshell. The net heat flux is calculated by combining convective, radiative and re-radiation phenomena using Sutton-graves(10), Tauber-Sutton(11) and Stephan-Boltzmann(12) relationships respectively. These equations are recollected below[3][11]:

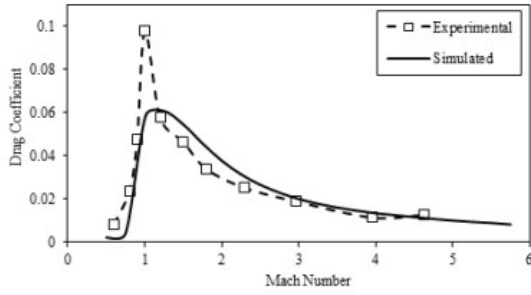


Figure 2: Drag coefficient (C_D) vs Mach number (M) for an arbitrary body: As suggested, C_D has an asymptotic behaviour as M increases[1]

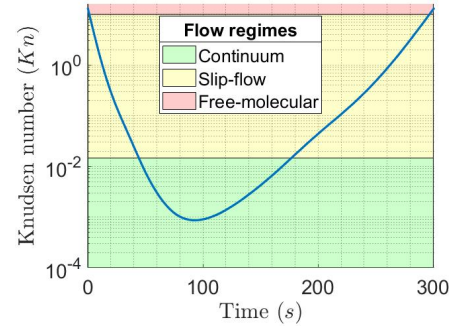


Figure 3: Variations in Knudsen number (Kn) along atmospheric pass

$$\dot{q}_{convective} = k \left(\frac{\rho}{R_N} \right)^{0.5} V^3 \text{ (Sutton-graves)} \quad (10)$$

$$\dot{q}_{radiative} = C_i R_n^a \rho_\infty^m f_i(V_\infty) \text{ (Tauber-Sutton)} \quad (11)$$

$$\dot{q}_{re-radiation} = \epsilon \sigma (T_W)^4 \text{ (Stephan-Boltzmann)} \quad (12)$$

$$\dot{q}_{Total} = \dot{q}_{convective} + \dot{q}_{radiative} \quad (13)$$

$$T_W = \sqrt[4]{\frac{\dot{q}_{Total}}{\epsilon \sigma}} \quad (14)$$

Where $R_N = 1.2$, which is the nose radius of the conic section of the aeroshell which was provided by dynamic structure team, $\epsilon = 5.67 \times 10^{-8}$ is the Stephen-Boltzmaan constant. The other parameters are related to the planet being analyzed, for Mars $a = 0.526$, $m = 1.2$, $\kappa = 1.9027 \times 10^{-4}$ and $C_i = 2.35 \times 10^4$. The remaining parameters, $f_i(V_\infty)$, are described in appendix D. Equation 14 was derived from simple heat flux balance, this will be a under-estimate but this value will only be used for optimization purposes rather than design purposes.

4 Sensitivity Analysis of Dynamic Models

4.1 Initial conditions

In order to determine a particular trajectory from the models introduced in section 2, an initial condition must be passed to the ODE solver to generate an initial value problem. This initial solution cannot be randomised, as each permutation of initial conditions will generate a different trajectory. The crucial question that must be asked is if such initial condition can be ever reached from an orbital mechanics point of view. This section focuses on determining the initial conditions for the three dimensional model. The state vector ($\mathbf{x}$) as well as what range of values each state can take are shown below:

$$\mathbf{x}_{6 \text{ states}} = [\delta \quad \lambda \quad R \quad V \quad \gamma \quad \chi]^T \quad (15)$$

Where $\delta \in [0, 2\pi]$, $\lambda \in [0, 2\pi]$, $R \in [R_{planet}, \infty]$, $V \in [0, \infty]$, $\gamma \in [-\pi/2, \pi/2]$ and $\chi \in [0, 2\pi]$. The initial condition physically represents the values of the states at Atmospheric interface (AI), this is the point where the spacecraft is considered to have entered significantly dense layers of the atmosphere. During literature review, the AI is defined as the location where the spacecraft starts experiencing an axial acceleration of 0.5 m/s^2 . Despite having said this, literature seems to disagree on a consistent AI altitude, Kumar and Tewari (2005) [8] use an AI altitude of 300Km where as Putnam and Braun (2014)[12] use an altitude of 150Km. An AI altitude of 125Km was selected, which corresponds to an axial acceleration of 0.005 m/s^2 , this altitude was in agreement with Fujita and Narita (2013) [6]. The altitude of the atmospheric interface does not largely influence the atmospheric trajectory, since increasing this altitude will simply make the dynamics model the last portion of the inserting hyperbolic orbit, where drag effects are negligible due to the low densities of the atmosphere.

The main job of Mission Design sub-team is Interplanetary trajectory design and optimization, the job of deciding what is the position and velocity vector, $\mathbf{r}$ and $\mathbf{v}$ respectively, as well as longitude (δ), latitude (λ) and heading angle (χ) was delegated to Mission Design sub-team, which are recollected in equation 16.



$$\mathbf{x}_{AI} = [0.58^\circ \quad 34.49^\circ \quad 125 \text{ Km} \quad 6.8917 \text{ Km s}^{-1} \quad \gamma_{AI} \quad -18.24^\circ]^T \quad (16)$$

The only parameters left to be determined is the flight path angle at AI (γ_{AI}), which will be calculated by the D-SAT and AsTRO algorithms.

4.2 Determination of Successful Aerocapture Trajectories (D-SAT) algorithm

The main function of the D-SAT algorithm is to perform a large number of “trial-and-error” trajectory simulations, for a single ballistic coefficient (β) by changing the flight path angle (FPA or γ) and repeating the process for every β in the range of interest. This procedure is performed in order to determine the range of FPAs that yield a successful aerocapture manoeuvre. Successful manoeuvres are differentiated from unsuccessful ones by computing the specific orbital energy of the post-aerocapture orbit, this is done by using equation 17. Where the velocity and position vector used are the vectors at Atmospheric Exit (AE), which is the point where atmospheric perturbations cease to perturb the orbit, this point has the same altitude as the AI. If the specific orbital energy is positive, this corresponds to an unbounded orbit and therefore it is flagged as unsuccessful, if ϵ is negative, this corresponds to a bounded orbit but not necessarily successful. The second condition that must be met in order to assure that the trajectory is successful, is to determine the semi-major axis (a) of post-aerocapture orbit. This can be done by using equation 17 and re arranging for a . If the semi major-axis yields an orbit with an apogee altitude bellow AI altitude, this corresponds to a failed aerocapture manoeuvre, since the spacecraft will crash into the planet.

All this data can be represented in a so called *Aerocapture corridor*, which aims to visualize the possible permutations of γ and β that yield a successful manoeuvre by plotting the flight path angle against ballistic coefficient, the final aerocapture corridor can be seen in figure 4. The aerocapture corridor highlights two regions of interest, non-robust and robust corridors. Due to atmospheric density uncertainty, the simulations had to be ran for different atmospheric models which included statistical deviation from normal, or standard deviations (σ). The non-robust corridor is the corridor yielded from simulating trajectories with ‘normal’ atmospheric densities, where as the robust corridor is the region robust to $\pm 3\sigma$ variations in atmospheric properties, which correspond to 99.7% of the scenarios.

$$\epsilon = \frac{|\mathbf{v}|^2}{2} - \frac{\mu}{|\mathbf{r}|} = -\frac{\mu}{2a} \quad (17)$$

Following these calculations, the D-SAT algorithm computes additional parameters in order to have a better insight on the dynamics. These parameters include; time and distance of atmospheric pass, orbital elements using an Orbital Element Determination algorithm, among others. Once these successful trajectories have been determined, the aerothermal environment will be characterised using equations presented in section 3.

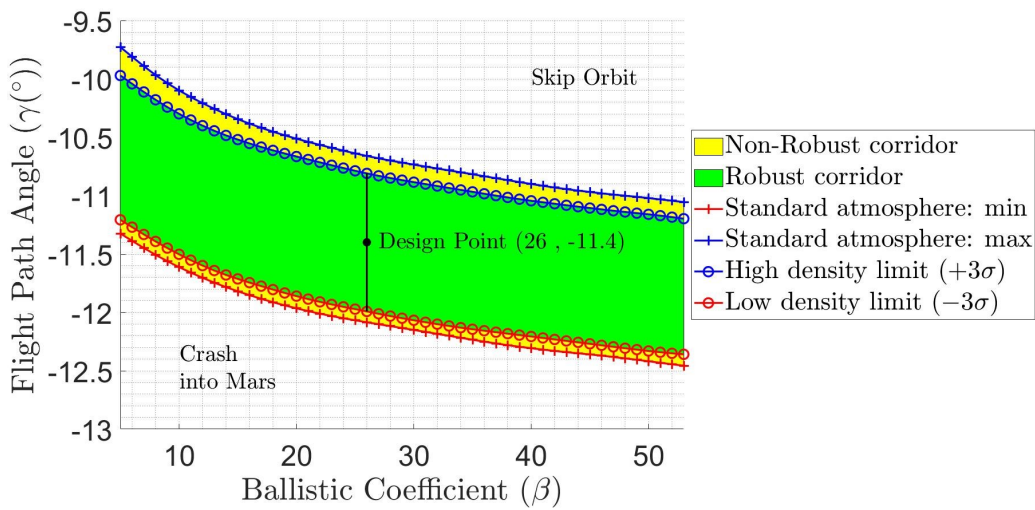


Figure 4: Aerocapture corridor: This corridor corresponds to a lifted re-entry ($L/D = 0.2$) with a $\pm 3\sigma$ atmospheric parameters uncertainty

The aerocapture corridor is used to set the design point. This point takes into consideration mission requirements and constraint conditions as well as physical limitations, like unfeasible aerodynamic properties or extreme accuracy. The primary design parameters are γ , β and L/D and can be determined from the aerocapture corridor, where as aerothermal properties cannot be extracted from this plot and must be calculated for each individual trajectory. The design of the aerodynamic shape of the aeroshell as well as configuration and



thickness of the TPS are carried forward by using the primary design parameters previously mentioned.

4.3 Sensitivity Analysis of Dynamics

Aeroassisted manoeuvres generally involve the use of a lifting body and precise guidance to compensate for many uncertainties, such as scattering of atmospheric density, errors in flight path angles at AI, erroneous dynamical models used to predict trajectories and variations of aerodynamics coefficients from predictions. This sections will analyze the dependency of the mission's realization with respect to the said uncertainties. This section is essential to impart some limitations on atmospheric interface coordinates. The results of this section will be relayed to inter-planetary trajectory design team, who will try and meet the preferred insertion coordinates while optimising their specific parameters

From this section onwards an analysis trimmed to a specific mission will be carried forward. This mission corresponds to the mission scenario 1 of the statement of work for an *AEROCAPTURE DEMONSTRATION MISSION - EXPRO PLUS* by ESA [4]. The details of the mission can be seen in table 1. From Mission Design's architecture trade-off study, a deployable heat shield will be used, this allows for a heat shield with dimensions larger than the launcher's fairing. In order for the heat shield to be larger than the fairing diameter, process 18 was used to determine what the ballistic coefficient of a heat shield with diameter equal to the launchers fairing would be, setting the upper bound of the analyzable β range, after rounding up to one significant figure, β equals to 50.

$$\beta = \frac{m}{C_D A} \quad \text{and} \quad A = \pi r^2 \quad \longrightarrow \quad \beta = \frac{400\text{Kg}}{1.55 \times \pi \times \left(\frac{2.6}{2}\right)^2} = 48.6 \quad (18)$$

Table 1: Mission specifications from ESAs statement of work [4]

Quantity	Value
Entry mass (m)	$\leq 400\text{Kg}$
Diameter in Launch configuration	$< 2,6\text{m}$
Hyperbolic excess inertial speed	$3.0 - 4.0\text{Km s}^{-1}$
Target operational orbit (after periapsis raise manoeuvre)	Eccentricity < 0.05 500-1000Km periapsis altitude Inclination $> 70^\circ$
Scientific payload	$\geq 20\text{Kg}$
Minimum FPA dispersion robustness	$\pm 0.4^\circ (\Delta\gamma \geq 0.8^\circ)$

4.3.1 Lift-to-Drag Ratio (L/D)

As previously mentioned, the design of an aerocapture capable spacecraft has certain limitations and constraints, one of them being the physically realisability of the system, with aerodynamics characteristics being the main constituent. A coefficient of special interest is the Lift-to-Drag ratio, the range which will be analyzed is $[0, 0.2]$, paying extra attention to the limits. This range encompasses values that are physically realisable, by either modulation of Angle of Attack (AoA) and/or by non-axisymmetric geometries, this is supported by legacy re-entry capsule technology like the Gemini and Apollo, which where able to achieve maximum L/D s of 0.25 and 0.35 respectively. Despite having developed more advanced re-entry aerodynamic shells, a deployable heat shield has only been tested once before, demanding a conservative approach. This means that (AoA) modulation will used as the primary methodology of increasing L/D instead of using a non-axisymmetric structure. For the given geometry a L/D of 0.2 can be achieved by an AoA of -18° . A comparison between a ballistic ($L/D = 0$) and a lifted re-entry ($L/D = 0.2$) corridor can be seen in figure 11, in appendix E.1.

Increasing the lift-to-drag ratio has a series of very important advantages, the most important being noticeable increments in the width of the corridor ($\Delta\gamma$), improving the robustness of the aerocapture manoeuvre to insertion FPA uncertainties. This is sustained by the fact that a ballistic entry, in combination with a $\pm 3\sigma$ scattering of atmospheric properties, will make an aerocapture manoeuvre completely unfeasible as the corridor has a negative width for low β . At low L/D s, $\Delta\gamma$ is very dependent on β , this can be seen from the top right plot in figure 11, as $\Delta\gamma$ steeply increases from a negative value towards zero, where the aerocapture corridor would open up, making the manoeuvre possible. As seen from the lifted re-entry simulations, bottom plots in figure 11, increasing L/D increases the aerocapture corridor width and reduced the dependencies on β , making the manoeuvre possible for a wide range of γ at every β . Even though $\Delta\gamma$ decreases for this scenario, it will never get to zero or negative values, this behaviour may be generate by low frequency noise introduced by the smoothing spline fits used to model the atmospheric parameters. Moreover, ODE solvers introduce random numerical noise to the solution which contributes to the non-monotonically decreasing property of the results. Moreover, lifted re-entries have other advantages like a more favourable aerothermal environment. This is be-



cause peak heating fluxes are reduced as well as g-forces enabling the use of lightweight TPS and supporting structures.

4.4 Drag Coefficient (C_D)

As mentioned in section 3, a simplistic aerodynamic model was used to simulate the forces experienced by the spacecraft during atmospheric pass. The model consisted of a constant C_D under the assumption of a monotonous flow physics, presumably continuum mechanics. However, from figure 3, monotonous flow physics cannot be assumed, as the flow over the spacecraft is well within the slip-flow regime for a large portion of the atmospheric pass and in Continuum regime for the remaining. Free-molecular flow is neglected. It must be noted that as soon as the flow leaves the Continuum regime, Navier-Stokes equations are no longer of use and the Boltzman equation must be solved employing Direct Monte Carlo Simulations (DMCS) in order to determine the flow properties at the Free-molecular regime. These regimes are differentiated with respect to the Knudsen number (Kn), defined in equation 19, which determines if continuum or statistical mechanics should be used to model the fluid. A transitional regime exists called the *slip-flow regime*, covering the range $0.0146 < Kn < 10$. The properties in this region are evaluated by using a logarithmic blending function between the two extreme regimes. Instead of solving Boltzman equations, a back-of-the-envelope approximation was performed, consisting of multiplying the drag coefficient at continuum flow by a factor of 1.35. This factor is in agreement with the Viking 1 mission flight data, where the mean C_D during Slip-flow territory was approximately 35% higher than the Continuum flow regime, see reference [2]. This section will attempt to quantify the effects of underestimating the C_D experienced during the manoeuvre.

In order to analyse the dependence of the mission's success with drag coefficient predictions, the robust aerocapture corridor yielded from a 35% increment in drag was superimposed with the initial drag predictions in figure 5. This yielded a smaller aerocapture corridor, reducing $\Delta\gamma$. However, this is still within feasible γ insertion accuracy ($\Delta\gamma > 0.4$). Drag uncertainties will only critically influence the design of a aerocapture capable spacecraft if the lift capabilities of said vehicle are low, in other words, if the L/D is in the vicinity of zero.

$$Kn = \frac{\lambda}{l} = \frac{\nu}{cl} \frac{\nu}{\sqrt{\frac{8RT}{\pi M}} l} \quad (19)$$

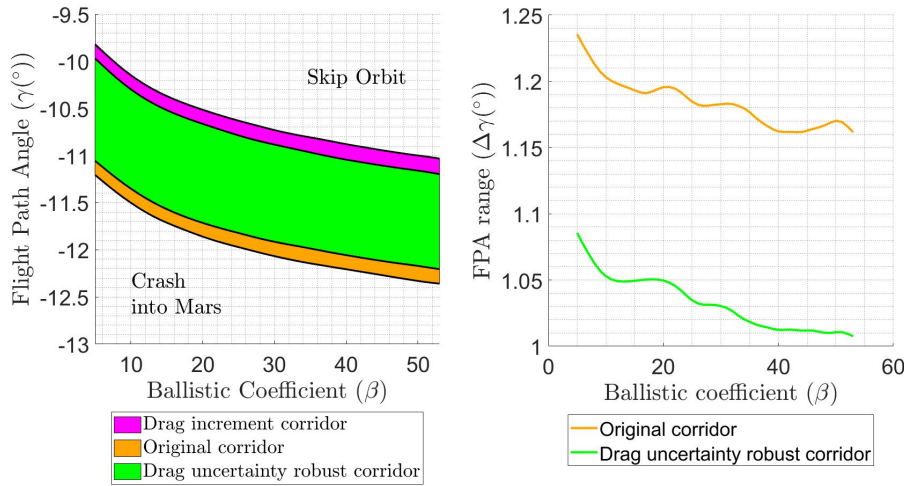


Figure 5: Drag uncertainty robust aerocapture corridor:

In order to account for the effects of slip-flow regime phenomena, the drag was increase by 35% and compared to the original aerothermal model. The drag uncertainty robust corridor is the region where the two different corridors (magenta and orange) overlap.

4.5 Scattering of Atmospheric Properties

The Atmospheric modelling of Mars and its complications fall outside of this report's scope and are analyzed in depth by Erikas in reference [9]. This being said, a brief summary of this analysis will be included in this report for completeness.

The width of the aerocapture corridor is known to be largely influenced by atmospheric properties. In order to determine a reliable atmospheric model with statistical measures, Mars Global Reference Atmospheric Model (Mars-GRAM) was used. This is explained in depth in reference [9]. In appendix E.2, a list of figures comparing different atmospheric models to different re-entry modes can be seen. As expected, increasing the



standard deviation of the model from $\pm 1\sigma$ to $\pm 3\sigma$ decreases the width of the aerocapture corridor ($\Delta\gamma$). In some cases, at significantly large L/D , the manoeuvre is still feasible well within insertion accuracy limitations ($\Delta\gamma > 0.4^\circ$), improving the robustness of lifted re-entries to insertion inaccuracies. However, for low L/D , $\Delta\gamma$ goes from an unfeasible value to a negative value, representing a non-realizable trajectory. Highlighting the importance of accurate atmospheric model in addition of precise and robust navigation and control systems that can deal with unpredictable day-of-flight atmospheric fluctuations.

5 Aerocapture (successful) Trajectory Robust Optimisation (AsTRO) algorithm

A comprehensive literature review shone a light on a large gap in the field of aerocapture design and trajectory optimisation [6][13]. This report aims to introduce a new optimisation methodology for primary design parameter selection (γ , β and L/D), laying the groundwork for the development of future optimisation procedures and algorithms that will make aerocapture a common feature of inter-planetary travel, introducing the AsTRO algorithm.

The AsTRO algorithm does not attempt to quantify the optimal γ for a specific β . Instead, it attempts to optimize β with respect to the features seen in equation 20 and selecting the γ which bisects the aerocapture corridor. This is done to increase the robustness of the manoeuvre to insertion inaccuracies and on-the-day atmospheric variations, introducing the *robustness* property of the algorithm.

5.1 Constraint Conditions

For the case study presented in section 4, the constraint conditions should be clearly quantified before examining the aerocapture corridor. Firstly, propellant mass needed for post-aerocapture orbital corrections must be minimized for this manoeuvre to be advantageous over traditional insertion burn manoeuvres, this performance driver will be quantified in terms of the total ΔV required for orbital corrections. Ideally, a maximum ΔV should be provided by the static structures team due to fuel tank volumetric limitations, but for a first iteration of this new optimisation methodology, quantifying this driver will suffice.

Secondly, as per the ESA statement of work brief[4] and the study carried forward in reference [6], with existing communication systems and Earth ground stations a limit to orbital determination accuracy exists. The fluctuation of flight path angle at AI is quantified as being as high as $\pm 0.2^\circ$. This means that the width of the aerocapture corridor at the optimised design point must be larger than 0.4° to account for insertion inaccuracies.

5.2 Performance drivers (Features)

Once these constraints have been considered, the system drivers that will contribute to the cost function must be carefully selected. According to SMAD [15], general system drivers for any space mission are: size (volume), on-orbit weight, power and data rate. The proposed optimisation methodology will address volumetric and on-orbit weight minimisation as well as aerocapture specific system drivers. The specific mission drivers are related to the areothermal environment experience during re-entry, which can be summarised in three quantities: Heat load (Q), Maximum heat flux ($\dot{q}_{max}$) and Maximum wall temperature (T_W). The cost states used in the optimization can be seen in equation 20.

$$\mathbf{x}_{\text{features}} = \begin{bmatrix} x(1) \\ x(2) \\ x(3) \\ x(4) \\ x(5) \\ x(6) \end{bmatrix} = \begin{bmatrix} V_{\text{structural}} \\ \Delta V \\ Q \\ \dot{q}_{\text{max}} \\ T_{W,\text{max}} \\ T_{\text{orbital}} \end{bmatrix} = \begin{bmatrix} \text{Volume of deployable structure} \\ \Delta V \text{ for orbital correction} \\ \text{Heat load} \\ \text{Maximum heat transfer rate} \\ \text{Maximum wall temperature} \\ \text{Orbital period} \end{bmatrix} \quad (20)$$

5.2.1 Volume of deployable structure ($V_{\text{structural}}$)

Equation 21 is used to compute the volume of the deployable structure from the ballistic coefficient. The derivation of said equation can be seen in appendix F.1.

$$V_{\text{structural}} = 1.3^2 \pi \left(\sqrt{\frac{\beta C_D \pi}{m}} - 1.3 \right) \quad (21)$$

Where symbols have their usual meaning.

5.2.2 ΔV budget for orbital correction and Orbital period (T_{orbital})

The ΔV budget will quantify the amount of fuel required to adjust the post-aerocapture orbit such that it meets the target operational orbit requirements listed in table 1. This manoeuvre is divided into two independent manoeuvres, a periapsis rise followed by circularization burn assuming a Hoffman transfer and a plane change. The derivation of the equations below as well as a diagram of the transfers can be seen in appendix F.2



$$\Delta V_{Hoffman} = \sqrt{\mu} \left[\sqrt{\frac{1}{a_2}} + \sqrt{\frac{2}{a_1(e_1+1)} - \frac{1}{\frac{1}{2}(a_1(e_1+1)+a_2)}} \right. \\ \left. - \sqrt{\frac{2}{a_2} - \frac{1}{\frac{1}{2}(a_1(e_1+1)+a_2)}} - \sqrt{\frac{2}{a_1(e_1+1)} - \frac{1}{a_1}} \right]$$

$$\Delta V_{Plane} = 2V_1|_a \sin\left(\frac{i_{target} - i_2}{2}\right)$$

$$\Delta V_{TOTAL} = \Delta V_{Plane} + \Delta V_{Hoffman} \quad (22)$$

Where i_{target} is the inclination of the target operational orbit, from table 1, $i_{target} \geq 70^\circ$. Where $V_1|_a$ is the velocity at apogee of the post-aerocapture orbit.

$$T_{orbital} = 2\pi \sqrt{\frac{a_1^3}{\mu}} \quad (23)$$

For orbital element reference see figure 15 in appendix F.2.

5.2.3 Aerothermal environment (Q , $\dot{q}_{max}$ and T_W)

Heat load is calculated using equation 24, the maximum heat flux is extracted from the heat flux profile of the trajectory using equations 10 to 13. However, in our application, where features will get normalized and scaled, being able to define the relationship without actually knowing the magnitude will suffice. The maximum heat flux follows the trend suggested by equation 25. The aeroshell maximum wall temperature is extracted from the wall temperature profile calculated using equation 14.

$$Q = \int_0^{t_f} \dot{q}_{Total} dt \propto \sqrt{\frac{\beta}{\sin(\gamma)}} \quad (24)$$

$$\dot{q}_{max} \propto \sqrt{\beta \sin(\gamma)} \quad (25)$$

5.3 Feature Scaling: Standarization

Initially, the idea behind the algorithm was to unify all the different drivers into a single efficiency measure which was mass. This was possible since most system drivers initially considered were able to be converted into mass. For example, ΔV could be converted into propellant mass with the help of the rocket equation. However, a key detail was overlooked: ΔV 's dependencies are not strictly limited to propellant mass, as it will have an influence on the fuel tank's volume, which in part increases structural mass as well, among others. In addition, to calculate the mass of the propellant, the mass of the spacecraft prior to the burn is necessary and this is an arbitrary value which would yield erroneous optima. On the other hand, where the sizing of the TPS is of concern, heat rates and loads as well as maximum wall temperatures will define the thickness of the aeroshell and its material, once the thickness and the nose radius is known, the mass of the aeroshell can be determined. Nevertheless, hidden dependencies will be lost in the process of converting thermal quantities into mass equivalent efficiency parameters. Optimising using the raw thermal quantities will also shine a light on what material is feasible for the TPS and for the inner structure. Instead of unifying all system drivers, also known as features, it was concluded that the raw features would be used to compute the "cost" of the design. This procedure was selected in order to avoid any optima which resulted in an unrealizable system due to hidden dependencies of the features.

Generally, when features do not have the same range of values, feature scaling must be applied if they want to be compared with each other since the cost function will penalise features with a larger magnitude than features with a small magnitude. The AsTRO algorithm will perform a special type of features scaling known as *Standardization*, where the mathematical operation can be seen in equation 26.

$$\mathbf{x}'_{Features} = \frac{\mathbf{x}(i) - \mu_i}{\sigma_i} \quad (26)$$

Where μ_i and σ_i represent the mean and standard deviation of the i^{th} feature in the feature vector ($\mathbf{x}_{Feature}$)

5.4 Importance of each feature

Currently, the algorithm gives equal importance to all feature, meaning that increments in deployable structure volume have the same impact to the trajectory optimisation as increments in maximum heat flux. This is



hardly true, as the success of the trajectory largely depends on the thermal load experienced during re-entry and barely on post-aerocapture orbit period. Adding weights to the optimization procedure allows the algorithm to prioritize the minimization of certain features over other less important ones. For example, if a weight of one and three are given to structural volume and maximum heat flux respectively, the algorithm will penalize, or increase the cost, of trajectories that experience a higher heat flux to trajectories that yield a longer orbit.

The criterion behind these values is the following; ΔV was largely penalized due to its impact on the feasibility of the mission over standard insertion burn manoeuvres as well as its hidden dependencies on volumetric and mass limitation. Maximum heat flux ($\dot{q}_{max}$) was also penalized as this parameter will significantly influence the sizing of the TPS. Finally, post-aerocapture orbit period will also be prioritized as long period orbits are undesirable. Low importance was awarded to the other features, note how maximum wall temperature is included in this set of features. In reality, maximum wall temperature should be heavily penalized as large $T_{W,max}$ will vaporize the TPS. This being said, low importance is given to this feature due to the lack of confidence placed in the method used to calculate this parameter. After careful considerations the weight granted to each feature are displayed in equation 27

$$\text{Weights}(k) = [1, 3, 1, 3, 1, 3]^T \quad (27)$$

5.5 Loss function architecture

A loss function is a measure of how optimal a trajectory truly is. Loss functions help to compare how well a trajectory has performed over other trajectories, it does so by comparing the features yielded by each trajectory with the desired, or target, features. An optimization process consists of minimizing this loss function and selecting the trajectory that yields the smallest possible loss, or cost. Essentially, the heart of the AsTRO algorithm is what loss function is used and how correctly it behaves under certain scenarios. For this case study, two different loss function were used and compared. Due to the minimization nature of the optimization, the target value of each feature is described in 28.

$$\mathbf{x}_{target} = \left[0, 0, 0, 0, 0, 2\pi \sqrt{\frac{a_{target}^3}{\mu}} \right]^T \quad (28)$$

The targeted features values do not necessarily need to have a physical interpretation. For example, it is well aware that volume or thermal loads cannot be equal to zero but they are desired to be kept close to zero, minimised. However, the orbital period of the post-aerocapture orbit is desired to be close to a certain value. This value is equal to the orbital period of the final operational orbit, with semi-major axis (a_{target}) equal to 4,390Km, which takes a value of 8,830s.

5.5.1 Least-Square method

The least-square method loss function is probably the most-popular loss function due to its simplicity and applicability to any optimization problem. However, as it will be seen later, this cost function is extremely sensitive to noise and outliers. The loss function is defined as:

$$J(\mathbf{x}) = \sum_i k(i) [\mathbf{x}(i) - \mathbf{x}_{target}(i)]^2 \quad (29)$$

5.5.2 Pseudo-Huber loss function

This method outperforms the least-square method by being aggressively convex in the vicinity of the minima and less steep at points far away from it, the pseudo-Huber loss function provides a smooth approximation of the Huber loss function. This loss function can be seen as a hybrid of the squared loss and absolute loss function, inheriting the best properties from each cost function. These are applying a square loss function close to the minima and transitioning to an absolute loss at extreme points, the transitioning regime is defined by the parameter δ which was tuned, with a final value equal to 0.01.

$$L_\delta(\mathbf{x}) = \delta^2 \left(\sqrt{1 + \sum_i \left[k(i) \left(\frac{(\mathbf{x}(i) - \mathbf{x}_{target}(i))^2}{\delta} \right) \right]} - 1 \right) \quad (30)$$

5.5.3 Loss function comparison

As previously mentioned, the key benefit of the Pseudo-Huber loss function over the Least-Square method is its steepness at the vicinity of minima, this behaviour can be clearly seen in figure 6. Due to the nature of the numeric ODE solver used, noise may be introduced in the results, which will change the optima for a non-aggressive convex function around the neighborhood of a minima, like the Least-Square method. In addition, neither loss function is completely robust to noise, further investigation in the field of noise robust optimisation

must be carried forward in order to find, or modify, a loss function capable of tolerating outliers. As per reference [16], Huber loss functions and function derived from this model present a relatively high tolerance to noisy data, this can be clearly seen in figure 7. Therefor more confidence is placed on the Pseudo-Huber loss function results due to its improved noise tolerance capabilities and pronounced convexity at minima regions.

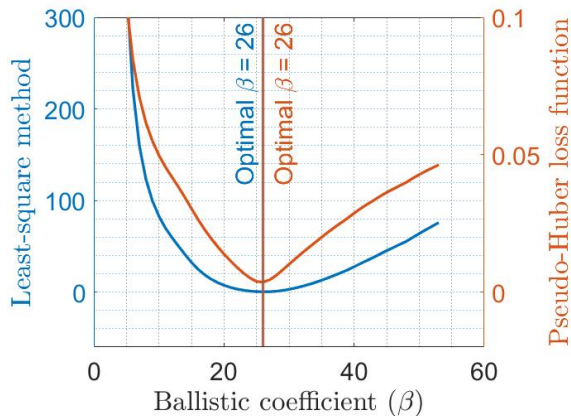


Figure 6: Loss function comparison

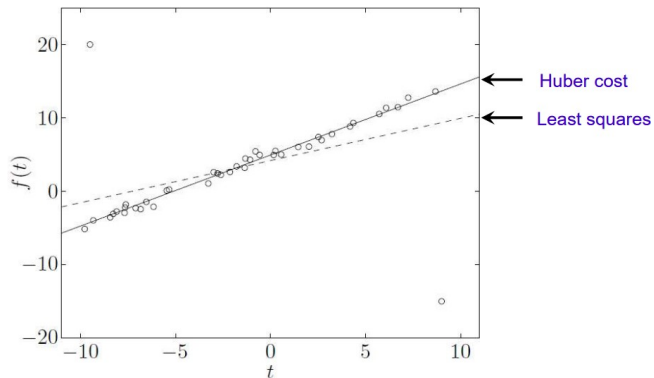


Figure 7: Noise robustness behaviour of loss functions

5.6 AsTRO Algorithm Sensitivity Analysis and Results

As analysed in section 4, the dynamics of the system where extremely sensitive to minor changes in some modelling parameters, this is also the case for the optimization of trans-atmospheric trajectories. In section 4, only the changes to the aerocapture corridor were considered, this means that only changes in β and γ could be measured. However, the D-SAT algorithm also evaluates the aerothermal environment and post-aerocapture orbital elements which where not used to compute the aerocapture corridor. Changes in the previously mentioned parameters would, therefore, not be noted in an aerocapture corridor plot. Whenever determining the cost of a design using the loss function, certain features where required. These included aerotehrmal quantities like Q , $\dot{q}_{max}$ and T_W as well as post-aerocapture orbital elements like ΔV which used semi-major axis (a) and inclination (i) of post atmospheric pass orbit. Changes in the aforementioned parameters will be noted by the AsTRO algorithm optimisation and will yield different optima. The scattering of these optima with respect to various deviations are summarised in tables 2 to 4.

Table 2: Weight vector (k) influence on optima

Weight vector (k)	Optimal Ballistic coefficient (β)	Optimal FPA ($\gamma(^{\circ})$)
[1, 1, 1, 1, 1, 1]	25	-11.38
[1, 3, 1, 3, 1, 3]	26	-11.40

Table 3: Atmospheric model influence on optima

Atmospheric model	Optimal Ballistic coefficient (β)	Optimal FPA ($\gamma(^{\circ})$)
Equatorial (Not appropriate)	28	-11.56
North pole (Not appropriate)	24	-11.80
South pole (Not appropriate)	28	-10.76
Appropriate model	26	-11.40

Table 4: Atmospheric interface location influence on optima

Type of insertion	Optimal Ballistic coefficient (β)	Optimal FPA ($\gamma(^{\circ})$)
Equatorial	28	-11.56
Polar (North)	27	-11.81
Polar (South)	30	-10.75
Actual insertion	26	-11.40

6 Design point

Any non-maneuvering re-entry trajectory and its corresponding aerothermal environment can be completely characterised by three parameters: β , γ and L/D . These parameters constitute the design point of the spacecraft and define the optimal trans-atmospheric trajectory. For a first iteration, a lift-to-drag ratio of 0.2 was assumed as this value broadens the aerocapture corridor significantly, increasing the corridor width ($\Delta\gamma$). As per table 5, the corridor width at the design point is roughly 1.18° , meaning that atmospheric interface flight path angle dispersions of up to $\pm 0.59^\circ$ can be tolerated. The design point not only takes into consideration uncertainties in insertion angle, it also considers the impact of low fidelity aerothermal models, variability of day-of-flight atmospheric properties and deviations in predictions of aerodynamic coefficient on the success of the mission. In order to mitigate against the aforementioned sources of variability, a wider aerocapture corridor of width 1.18° was desirable instead of simply matching the minimum corridor width stated in ESA's petition of work brief [4], which corresponds to a corridor width of 0.8° .

Table 5 contains the final design parameters of the spacecraft, the trans-atmospheric trajectory properties and the post-aerocapture orbital elements.

Table 5: Final design parameters, trajectory properties and post-aerocapture orbital elements

Quantity	Value
Optimal Ballistic coefficient (β_{des})	26
Optimal Insertion flight path angle (γ_{des})	-11.40°
Aerocapture corridor width at design point ($\Delta\gamma_{des}$)	1.18°
Design Lift-to-Drag ratio (L/D_{des})	0.20
Trimmed Angle of attack (α_{des})	-18.00°
Trajectory heat load (Q)	53.20 MJ
Trajectory maximum heat flux ($\dot{q}_{max}$)	75.72 W cm^{-2}
Trajectory maximum axial acceleration ($a_{n,max}$)	17.90 ms^{-2}
Specific orbital energy (ϵ)	-5.92×10^6
Apogee altitude (h_a)	1013.70Km
Eccentricity (e)	0.22
Inclination (i)	48.70°

7 Future Mission Design

As this mission was focused on the demonstration of aerocapture manoeuvres, restrictions were placed on the class of re-entry trajectory, limiting us to ballistic or lifted re-entries. The next step would be to increase the complexity of the re-entry trajectory, which would come with associated improvements in performance, at a higher design cost, stemming from a much longer design process. This increase in complexity comes from incorporating lift vectoring capabilities to the re-entry spacecraft. Augmented lift capabilities can be met by using wings, such as in NASA's Space Shuttle or Boeing's X-37 prototype, or by using a more structurally robust lifting body in addition to precise attitude control systems. Certain restrictions exist regarding lift-to-drag ratios of hypersonic lifting bodies, usually setting an upper bound limit at unity. These bodies are able to largely modify their trajectory by actively vectoring their lift, which can be accomplished by using strategically placed thrusters. However, the control of said class of trajectories is associated with very accurate Inertial Mass Units (IMU) and complex Data Handling Units (DHU), as well as the previously mentioned thrusters, which would greatly increase the weight of the spacecraft.

This being said, maneuvering re-entries introduce a vast list of advantageous phenomena at the cost of their controllability complexity. Firstly, additional speed can be shed through manoeuvring. Because of the flatter shape of a manoeuvring entry vehicle, the lift generation leads to a greater amount of drag due to skin friction. Which increases the rate of heat transfer experienced by the aeroshell, requiring a shallower atmospheric dive and rapid velocity losses through banking manoeuvres. The Space Shuttle used such manoeuvring techniques by performing sharp *S-turns*.

A key property of this class of trajectories is commonly overlooked. Using active lift vectoring, the trans-atmospheric trajectory can be shaped such that the post-atmospheric pass orbit has an increased, or decreased, inclination compared to the pre-atmospheric pass orbit. This allows for a significant reduction in ΔV associated to plane change manoeuvres although taking longer than a propelled change. Due to all the complications and dependencies related to manoeuvring entries, the optimisation and determination of said trajectory would quickly become an optimal guidance problem, setting the stage for future investigation in this field.



References

- [1] A. S. M. AL-OBAIDI and J. X. Y. MOO. Effect of atmospheric altitude on the drag of wing at subsonic and supersonic speeds. *Journal of Engineering Science and Technology*, 71(83), May 2017.
- [2] R. C. Blanchard and G. D. Walberg. Determination of the hypersonic-continuum/rarefied-flow drag coefficient of the viking lander capsule 1 aeroshell from flight data. Technical report, Dec 1980.
- [3] P. Bruce. Aerothermodynamics of launch and re-entry vehicles, chapter 1: Hypersonic aerothermodynamics, 2021.
- [4] ESA. aerocapture demonstration mission - expro plus esa-e3p-marsx-sow-003. Technical report, 2021.
- [5] G. Falcone, J. W. Williams, and Z. R. Putnam. Assessment of aerocapture for orbit insertion of small satellites at mars. *JSR Spacecraft / Rockets*, 56(6), May 29, 2019.
- [6] K. Fujita and S. Narita. Conceptual study of a small-sized mars aerocapture demonstrator. Technical report, 2013.
- [7] A. Knoll and D. Amato. Spacecraft systems lecture 5 - launch vehicle design, Jan 26, 2022.
- [8] M. Kumar and A. Tewari. Trajectory and attitude simulation for aerocapture and aerobraking. *Journal of Spacecraft and Rockets*, 42(4), 2005.
- [9] E. Kymantas. Flight-control architecture analysis atmospheric driven trajectory study with mars-gram 2010. Technical report, June 2022.
- [10] C. Namuroy. Preliminary aerothermal analysis of the deployable heat shield for aegis. Technical report, June 2022.
- [11] NASA. Aerothermodynamics course, lecture 1 - stagnation point heating.
- [12] Z. R. Putnam and R. D. Braun. Drag-modulation flight-control system options for planetary aerocapture. 51(1):Journal of Spacecraft and Rockets, 2014.
- [13] S. J. Saikia, H. Saranathan, M. J. Grant, and J. M. Longuski. Trajectory optimization for adaptive deployable entry and placement technology (adept). In *AIAA/AAS Astrodynamics Specialist Conference*, San diego, CA, 4-7 August 2014 2014.
- [14] K. Sutton and M. E. Tauber. Stagnation-point radiative heating relations for earth and mars entries. *Journal of Spacecraft and Rockets*, 28(1), 2012.
- [15] J. R. Wertz, D. F. Everett, and J. J. Puschell. *Space Mission Engineering: The New SMAD*. Microcosm Press, Hawthorne, CA 90250 USA, second edition, 2015.
- [16] A. Zisserman. B1 optimization module, lecture 4 - convexity, robust cost functions and optimizing non-convex functions.



C Orbital Element Determination Algorithm

Algorithm 1 Calculate Post Aerocapture Orbital Elements

- 1: **INPUT:** $\mathbf{r}$ and $\mathbf{v}$ at Atmospheric Exit (AE)
 - 2:
 - 3: Calculate specific angular momentum: $\mathbf{h} = \mathbf{r} \times \mathbf{v}$
 - 4: Calculate Semi-major axis: $a = \frac{\mu}{2} \left[\frac{|\mathbf{v}|^2}{2} - \frac{\mu}{|\mathbf{r}|} \right]^{-1}$
 - 5: Calculate eccentricity vector: $\mathbf{e} = \frac{\mathbf{v} \times \mathbf{h}}{\mu} - \frac{\mathbf{r}}{|\mathbf{r}|}$
 - 6: Calculate orbit inclination: $\arccos\left(\frac{h_z}{|\mathbf{h}|}\right)$
 - 7: Calculating location of the node line: $\bar{\mathbf{n}} = [0, 0, 1]^T \times \mathbf{h} = [-h_y, h_x, 0]^T$
 - 8: **if** $n_y \geq 0$ **then**
 - 9: $\Omega = \arccos\left(\frac{n_x}{|\mathbf{n}|}\right)$
 - 10: **else**
 - 11: $\Omega = 2\pi - \arccos\left(\frac{n_x}{|\mathbf{n}|}\right)$
 - 12: Calculate argument of periapsis:
 - 13: **if** $e_z > 0$ **then**
 - 14: $\omega = \arccos\left(\frac{\mathbf{n} \cdot \mathbf{e}}{|\mathbf{n}||\mathbf{e}|}\right)$
 - 15: **else**
 - 16: $\omega = 2\pi - \arccos\left(\frac{\mathbf{n} \cdot \mathbf{e}}{|\mathbf{n}||\mathbf{e}|}\right)$
 - 17: Calculate True anomaly: $\nu = \arccos\left(\frac{\mathbf{e} \cdot \mathbf{r}}{|\mathbf{e}||\mathbf{r}|}\right)$
 - 18:
 - 19: **RETURN:** Orbital elements = $[a, |e|, i, \Omega, \omega, \nu]^T$
-



D Thermal model constants for Martian atmosphere

In equation 11, a atmospheric specific function is required to determine the radiation heat flux on the spacecraft. This function is defined in the figure below. This function is only defined for a range of velocities, when the velocity decreases below 6000m^{-1} , the function is defined as taking a value of zero, ideally neglecting photonic phenomena in the boundary layer [14].

$V, \text{ m/s}$	$f_M(V)$
6000	0.2
6150	1.0
6300	1.95
6500	3.42
6700	5.1
6900	7.1
7000	8.1
7200	10.2
7400	12.5
7600	14.8
7800	17.1
8000	19.2
8200	21.4
8400	24.1
8600	26.0
8800	28.9
9000	32.8
---	---
---	---

Figure 10: Mars specific function ($f_i(V)$) for radiative heat flux modelling



E Aerocapture sensitivity figures

In section 4, the sensitivity of the aerocapture corridor and its width ($\Delta\gamma$) was studied. In this section the figures related to this analysis can be examined

E.1 Comparison between Ballistic and Lifted re-entries

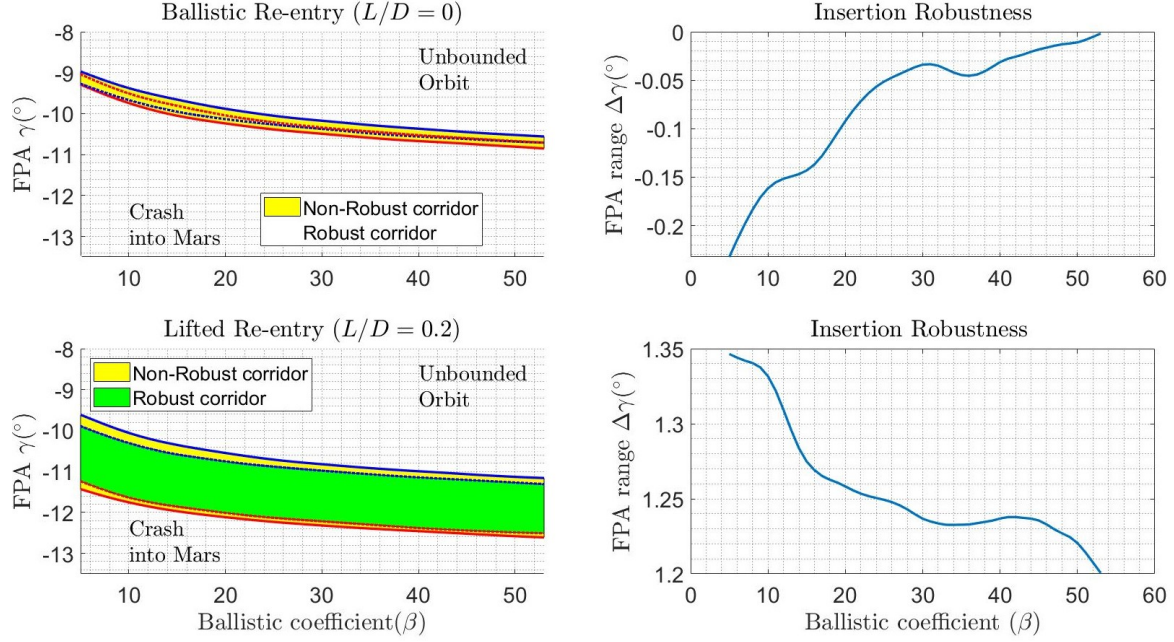


Figure 11: Aerocapture corridor sensitivity to Lift-to-Drag ratio and comparison of corridor width

E.2 Atmospheric model variations

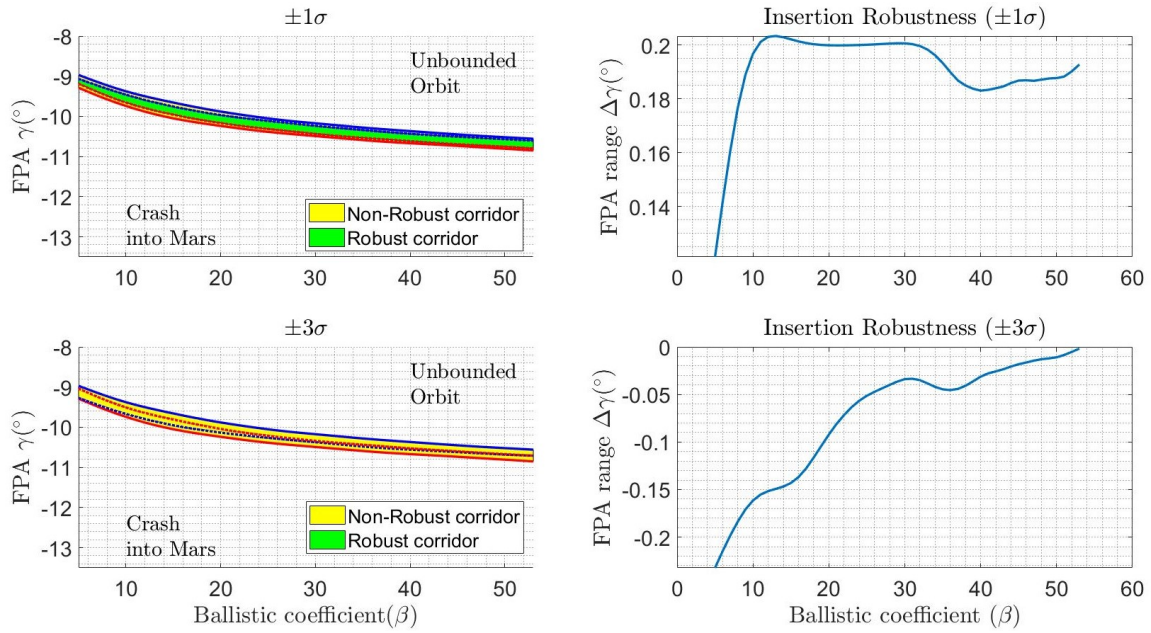


Figure 12: Ballistic Re-entry corridor: Sensitivity to scattering of atmospheric properties

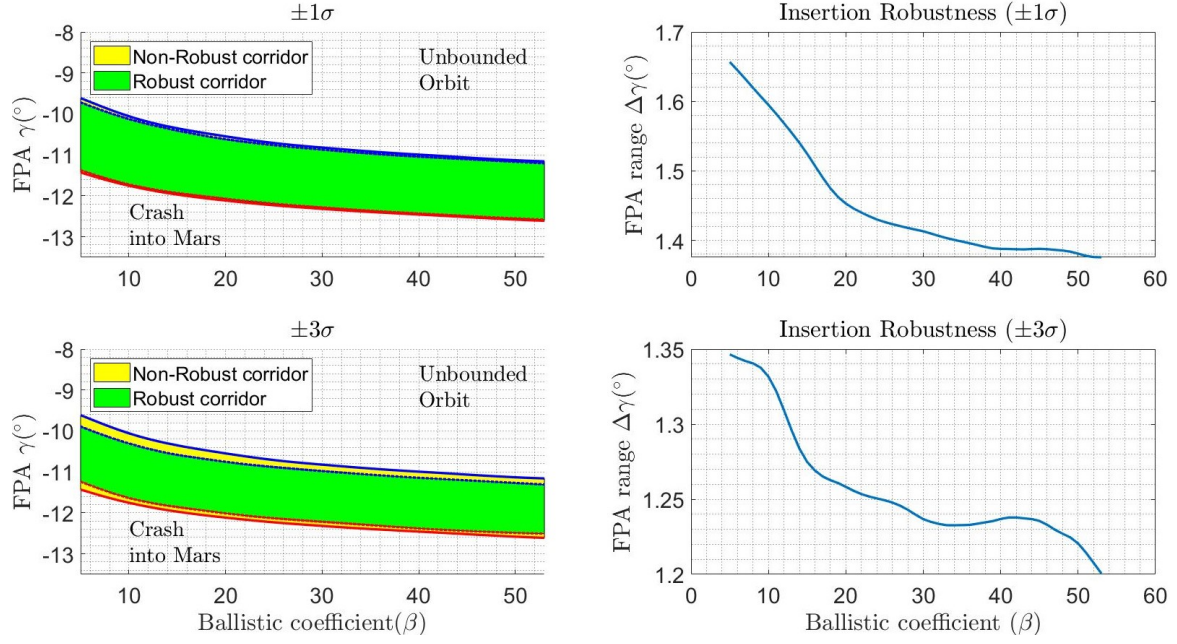


Figure 13: Lifted Re-entry corridor: Sensitivity to scattering of atmospheric properties

F Derivation of performance drivers used in cost function

F.1 Volume of dynamical structure

$$\beta = \frac{m}{C_D A} = \frac{m}{C_D \pi r^2}$$

$$r = \sqrt{\frac{m}{\beta C_D \pi}}$$

$$V_{umbrella} = 1.3^2 \pi [r - 1.3] = 1.3^2 \pi \left(\sqrt{\frac{m}{\beta C_D \pi}} - 1.3 \right)$$

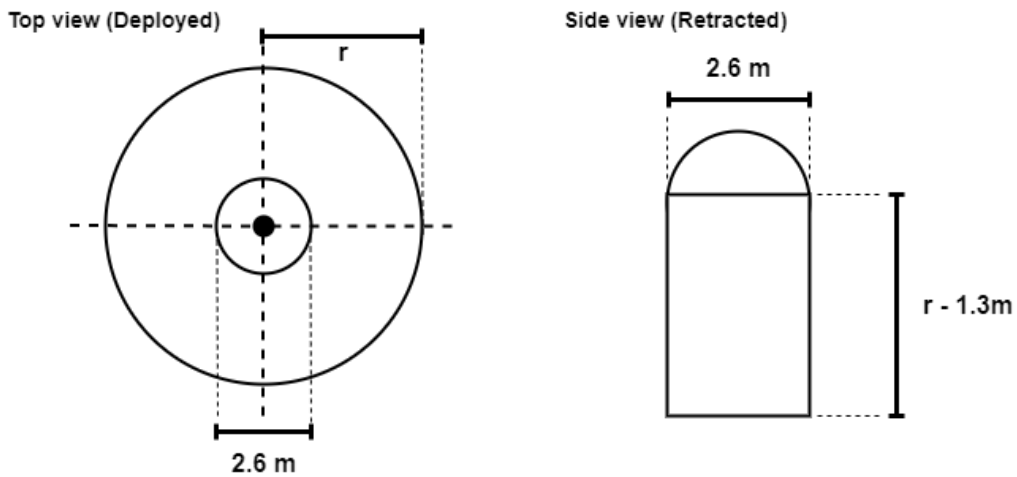


Figure 14: Retracted and deployed visualization of deployable structure

F.2 Post-aerocapture orbital correction manoeuvres

$r_{a|1} = a_1 (e_1 + 1)$, apogee radius of post aerocapture orbit

$a_t = \frac{1}{2} (r_{a|1} + a_2)$, Semi-major axis of transfer orbit

$V_{1|a} = \sqrt{\mu \left(\frac{2}{r_{a|1}} - \frac{1}{a_1} \right)}$, velocity at apogee of post aerocapture orbit

$V_{t|a} = \sqrt{\mu \left(\frac{2}{r_{a|1}} - \frac{1}{a_t} \right)}$, Velocity at apogee of transfer orbit

$\Delta V_{|1} = V_{t|a} - V_{1|a}$, First burn

$V_{t|p} = \sqrt{\mu \left(\frac{2}{a_2} - \frac{1}{a_t} \right)}$, Velocity at perigee of transfer orbit

$V_{2|p} = \sqrt{\frac{\mu}{a_2}}$, Velocity at perigee of target orbit

$\Delta V_{|2} = V_{2|p} - V_{t|p}$, Second burn

$\Delta V_{Hoffman} = \sqrt{\mu} \left[\sqrt{\frac{1}{a_2}} + \sqrt{\frac{2}{r_{a|1}} - \frac{1}{a_t}} - \sqrt{\frac{2}{a_2} - \frac{1}{a_t}} - \sqrt{\frac{2}{r_{a|1}} - \frac{1}{a_1}} \right]$
, Total Hofmann transfer burn

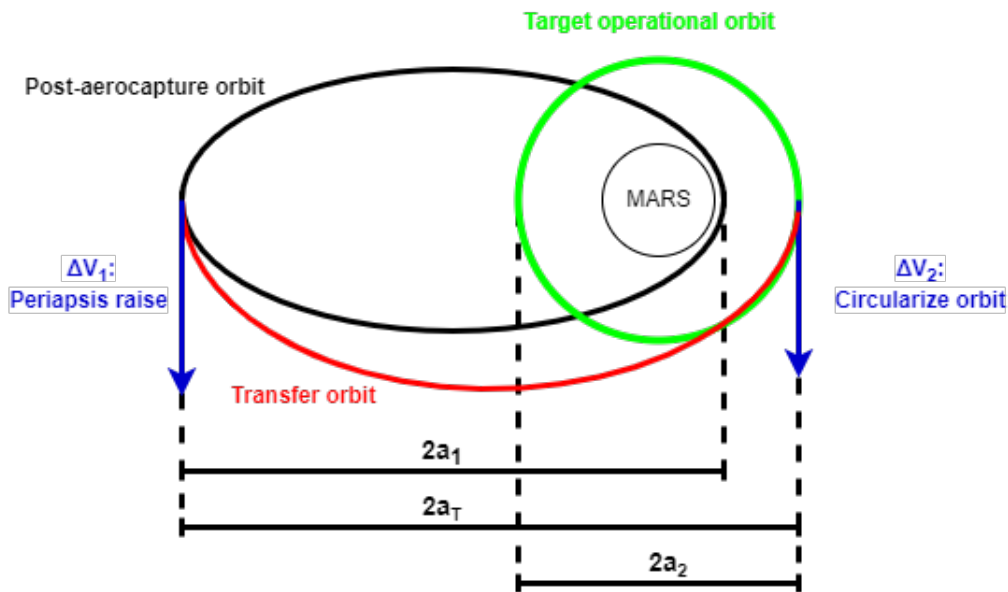


Figure 15: Periapsis raise and Circularization manoeuvre

G MATLAB Code: D-SAT Algorithm

```

1
2
3 %-----DETERMINATION OF SUCCESSFUL AEROCAPTURE TRAJECTORIES (D-SAT) ALGORITHM-----
4
5
6 %% HOUSEKEEPING
7
8 global Rm mu h_AI m Omega L_D L T_w_start models vinf CD
9
10 %% LOADING DATA
11
12 load('Scenario1.mat') % ESa brief scenario 1 data
13
14 % Load atmospheric property models

```



```
15 load('MarsGRAMtemperature.mat')
16 load('MarsGRAMpressure.mat')
17 load('MarsGRAMmolarwgt.mat')
18 load('MarsGRAMdensity.mat')
19 load('MarsGRAMdensity_hi3sd.mat')
20 load('MarsGRAMdensity_lo3sd.mat')
21
22 models.MarsGRAMdensity = MarsGRAMdensity;
23 models.MarsGRAMdensity_hi = MarsGRAMdensity_hi3sd;
24 models.MarsGRAMdensity_lo = MarsGRAMdensity_lo3sd;
25 models.MarsGRAMtemperature = MarsGRAMtemperature;
26 models.MarsGRAMpressure = MarsGRAMpressure;
27 models.MarsGRAMmolarwgt = MarsGRAMmolarwgt;
28
29 %% ----- CONSTANTS -----
30
31 Rm = 3389.5e3; % Radius of Mars
32 mu = 4.282837e13; % Mars Gravitational Parameter
33 m = S1.mass; % Mass of the spacecraft, in agreement with ESA brief scenario 1
34 Omega = 2*pi / (24*3600 + 38*60 + (22+35)/2); % Rotationa period of Mars
35 L_D = 0.2; % Lift to drag ratio
36 L = 2.6; % Characteristic length of the spacecraft
37
38 vinf = 3.5; % From Inter-planetary mission design (Km/s)
39
40 % Atmospheric interface (AI)
41
42 h_AI = 125e3; % Altitude of AI
43 r_AI = Rm+h_AI; % Position of AI
44 v_AI = V_hype_fun(r_AI); % Velocity at AI
45
46
47 %% ----- ATMOSPHERIC TRAJECTORY SIMULATION -----
48
49 plot_option = false;
50 density_analysis = true;
51 save_option = true;
52
53 if save_option && (~density_analysis)
54     density_analysis = true;
55 end
56
57 gamma_range = -13.5:0.05:-8; % From multiple simulations, this range encompass all possible
    FPAs
58 BC_range = 5:1:53; % A single BC to be used to study jettison affects (area change)
59
60 architecture = 2; % 1 = Single Jettison, 2 = Constant ballistic coefficient,
61 architecture_label = {'single_jet','ballistic'};
62
63 params = [v_AI]; % Vector of parameter passed to trajectory_simulation_3D.m
64
65 switch architecture
66     case 2
67         S = trajectory_simulation_3D(gamma_range, BC_range, params, plot_option,
            density_analysis);
68     case 1
69         BC_jet=20; % Arbitrary post discrete single event drag modulation event
70         S = trajectory_simulation_jettison(gamma_range, BC_range, params, BC_jet);
71     otherwise
72         disp('Enter an architecture type')
73 end
74
75 %%----- POST TRIAL AND ERROR SIMULATION DATA PROCESSING AND DISPLAY
    -----
76
77 % Load structure that contains all the data from trial and error simulations, corresponding to
    successful trajectories
78 A = S.(architecture_label{architecture}).(['BC' num2str(BC_range(1))]).normal;
79
80 % Load Atmospheric properties specific for succesful trajectories
81 [~, ~, ~, Y] = marsatm(h,1, false, A.array1);
82
83 var_names = {'Flight path angle', 'Specific energy', 'eccentricity', 'Inclination', 'True anomaly
    (deg)', 'Atmospheric time pass (s)', 'Distance atmospheric pass (Km)'};
84
```



```
85 T1 = table(A.gamma,A.specific_energy,A.e,rad2deg(A.i),(A.nu),A.time_atm_pass,A.  
86 distance_atm_pass,'VariableNames', var_names);  
87  
88 AeroThermoTable = array2table(Y);  
89 AeroThermoTable.Properties.VariableNames = ["Temperature (K)","Density $(kg//m^3)$", "Mach", "  
90 Knudsen Number","Speed of Sound (m/s)","Flow Regime","Dynamic Viscosity","Reynolds Number  
91 ","Thermal Conductivity","Time (s)"];  
92 disp(AeroThermoTable)  
93  
94 %% ----- FLIGHT CORRIDOR PARAMS -----  
95  
96 if length(BC_range)>1  
97     lb=zeros(3,length(BC_range)); % lower bound flight path angles stored here  
98     ub=zeros(3,length(BC_range)); % upper bound flight path angles stored here  
99  
100     for i = 1 : 3  
101         for index = 1:length(BC_range)  
102             lb(i, index)=min(S.(architecture_label{architecture}).(['BC' num2str(BC_range(  
103 index))]).(density_label{i}).gamma);  
104             ub(i, index)=max(S.(architecture_label{architecture}).(['BC' num2str(BC_range(  
105 index))]).(density_label{i}).gamma);  
106         end  
107     end  
108  
109     normal_lb = fit(BC_range',lb(1,:),'smoothingspline','SmoothingParam',0.07);  
110     normal_ub = fit(BC_range',ub(1,:),'smoothingspline','SmoothingParam',0.07);  
111     low_lb = fit(BC_range',lb(2,:),'smoothingspline','SmoothingParam',0.07);  
112     low_ub = fit(BC_range',ub(2,:),'smoothingspline','SmoothingParam',0.07);  
113     high_lb = fit(BC_range',lb(3,:),'smoothingspline','SmoothingParam',0.07);  
114     high_ub = fit(BC_range',ub(3,:),'smoothingspline','SmoothingParam',0.07);  
115  
116     lb = [normal_lb(BC_range)'; low_lb(BC_range)'; high_lb(BC_range)'];  
117     ub = [normal_ub(BC_range)'; low_ub(BC_range)'; high_ub(BC_range)'];  
118  
119     FlightCorridor.BC = BC_range;  
120     FlightCorridor.lower_bound = lb;  
121     FlightCorridor.upper_bound = ub;  
122     if ub(2,:)-lb(3,:) > 0  
123         FlightCorridor.delta_gamma = abs(ub(2,:)-lb(3,:));  
124     else  
125         FlightCorridor.delta_gamma = 0;  
126     end  
127     FlightCorridor.runs = S.ballistic;  
128  
129     if save_option  
130         if L_D == 0  
131             Ballistic_corridor_states6 = FlightCorridor;  
132             save('Ballistic_corridor.mat','Ballistic_corridor_states6');  
133         elseif L_D == 0.1  
134             Lifted_corridor_1_states6 = FlightCorridor;  
135             save('Lifted_corridor_01.mat', "Lifted_corridor_1_states6");  
136         elseif L_D == 0.2  
137             Lifted_corridor_2_states6 = FlightCorridor;  
138             save('Lifted_corridor_02.mat', "Lifted_corridor_2_states6");  
139         end  
140     end  
141 end  
142 end  
143 end
```

G.1 MATLAB Code: Trajectory simulations

```
1  
2 function [Structure] = trajectory_simulation_3D(gamma_range, BC_range, params, plot_option,  
3 density_analysis)  
4 %% INFORMATION  
5 % Calling this function runs trajectory simulation with current aerothermodynamic  
6 % model and plots a series of plots , in addition it outputs a structure  
7 % with key information  
8 % INPUTS: -gamma_range: [min, max] range of flight path angles which  
9 % will be analyzed (deg)  
10 % -BC_range: [min, max] range of ballistic coefficients which  
11 % will be analyzed (kg/m2)  
12 % -params: array which contains parameters essential for simulations
```




```
12 %   OUTPUTS:      -Structure: structure that contains useful data of successful trajectories,
13 %   such
14 %   as;
15 %               -> specific_energy = specific orbital energy of resulting orbit
16 %               -> a = semi-major axis
17 %               -> nu = True anomaly
18 %               -> e = eccentricity of final orbit
19 %               -> i = inclination (rad)
20 %               -> Omega = longitude of the ascending node (rad)
21 %               -> omega = argument of periapsis (rad)
22 %               -> gamma = vector that contains all gammas of successful manoeuvres
23 %               -> final_position: [x, y, z] coordinates of atmospheric exit
24 %               -> final_velocity: vector of velocity at atmospheric exit
25 %               -> time_atm_pass: atmospheric pass time
26 %               -> distance_atm_pass: atmospheric pass distance
27 % PLOTTING PARAMETERS
28 Markers = {'.', '+', 'x', '*', 'o', 'v', 'd', '^', 's', '>', '<'};
29 Line_format={'-', '--', ':', '-.'};
30 Color={'b', 'k', 'r', 'm', 'g', [255, 165, 0]./255};
31 Markermax=length(Markers);
32 Line_formatmax=length(Line_format);
33 Colormax=length(Color);
34
35
36 global Rm mu h_AI
37
38 v_AI = params(1);
39
40 % ODE solver parameters
41 tspan = [0, 700]; % Time span being analysed
42
43 if length(gamma_range) == 1
44     Opt = odeset('Events', @myEventLimit, 'RelTol',1e-2,'AbsTol',1e-2,'MaxStep',01); %
45     Solver option which terminates run when R state reached atmospheric boundary or martian
46     surface
47 else
48     Opt = odeset('Events', @myEventLimit, 'RelTol',1e-2,'AbsTol',1e-2);
49 end
50
51 %% ODE initial conditions (for final insertion)
52 delta0 = deg2rad(0.5798); % Initial Longitude, +ve to east
53 lambda0 = deg2rad(34.49); % Initial Latitude, +ve northern hemisphere
54 R0 = Rm + h_AI; % Atmospheric interface distance from martian centre
55 V0 = v_AI; % Atmospheric interface velocity
56 chi0 = deg2rad(90-108.24); % heading angle
57
58 % Pre-allocating variable for speed
59
60 specific_energy = zeros(length(gamma_range),1);
61 time_atm_pass = zeros(length(gamma_range),1);
62 distance_atm_pass = zeros(length(gamma_range),1);
63
64 fprintf('\n ----- SIMULATION RUNNING ----- \n')
65
66 for BC = BC_range % Looping BC range
67
68     fprintf('CURRENT CB: %d \n', (BC))
69
70     % Pre-allocating variables for speed
71     final_position = zeros(size(gamma_range,2),3);
72     final_velocity = zeros(size(gamma_range,2),3);
73
74     % Density analysis
75
76     if density_analysis
77         density_label = {'normal', 'low', 'high'};
78     else
79         density_label = {'normal'};
80     end
81
82     for density_mode = 1:length(density_label)
83         tic
84
85         % Trackers
86         gamma_tracker = 1;
```



```
85     array_tracker = 1;
86
87     fprintf('\nDENSITY SCENARIO: %s \n', density_label{density_mode})
88
89     if plot_option && length(gamma_range)~=1
90         figure('units','normalized','outerposition', [0 0 1 1])
91     end
92
93     for gamma0 = gamma_range
94
95         x0 = [delta0, lambda0, R0, V0, deg2rad(gamma0), chi0]; % ODE initial conditions
96
97         % Calling stiff ODE solver
98         [t, y] = ode23s( @(t, x) reentry_dynamics(t, x, BC, density_mode), tspan, x0, Opt)
99
100
101         Omega = 2*pi / (24*3600 + 38*60 + (22+35)/2);
102
103         % ODE solver solution processing
104         delta = y(:,1)+t.*Omega;
105         lambda = y(:,2);
106         R = y(:,3);
107         h = R- Rm;
108         V = y(:,4);
109         gamma = y(:,5);
110         chi = y(:,6);
111         time_atm_pass(gamma_tracker) = t(end);
112         distance_atm_pass(gamma_tracker) = trapz(t,V);
113
114         % Thermal model
115         [q_dot, q_dot_max, Q] = thermalModelSegu(t, R - Rm, V, 1, density_mode);
116
117         if plot_option && length(gamma_range)~=1
118             clf
119             sgtitle(['Ballistic coefficient (\beta) = ', num2str(BC)], 'fontSize', 20)
120
121             subplot(2,2,[1, 2])
122
123             yyaxis left
124             plot(t,h/1e3,'b')
125             ylabel('Altitude (Km)')
126             yline(0, '-.b', 'SURFACE', 'LineWidth', 2)
127             yline(h_AI/1e3, '-.b', 'ATMOSPHERIC INTERFACE', 'linewidth', 2)
128             ylim([-10, h_AI/1e3+20])
129
130             yyaxis right
131             plot(t,V/1e3,'r')
132             ylabel('Velocity (Km/s)')
133             ylim([0,inf])
134
135             title(['\gamma_0 = ', num2str((gamma0))], 'FontSize', 20)
136             xlabel('Time (s)')
137             xlim([0,10*60])
138
139             legend({'Altitude', '', 'Velocity'}, 'Location', 'east', 'FontSize', 18)
140             subplot(2,2,3)
141             traj_plot_3D([t, R, V, gamma, delta, lambda, chi], plot_option);
142
143             drawnow
144         end
145         % ----- AXIAL ACCELERATION -----
146
147         if h(end) > h_AI
148             specific_energy(gamma_tracker) = 0.5*V(end)^2 - mu/R(end);
149             if specific_energy(gamma_tracker) < 0
150                 %% AXIAL ACCELERATION
151                 an = axial_acceleration([R, V, gamma, lambda, chi], BC);
152
153                 Structure.ballistic.(['BC' num2str(BC)]).(density_label{density_mode}).(['
array' num2str(array_tracker)]) = [t, R, V, gamma, delta, lambda, chi, q_dot, Q, [
q_dot_max; zeros(size(Q,1)-1, size(Q,2))], an];
154                 array = Structure.ballistic.(['BC' num2str(BC)]).(density_label{
density_mode}).(['array' num2str(array_tracker)]);
155
156                 [final_position(gamma_tracker,:), final_velocity(gamma_tracker,:)] = ...
```



```
157         traj_plot_3D(array, false);
158
159         array_tracker = array_tracker+1;
160     end
161 else
162     specific_energy(gamma_tracker) = 0;
163 end
164
165 fprintf(' ')
166
167 gamma_tracker = gamma_tracker +1;
168
169 end
170
171
172 if plot_option && length(gamma_range)~=1
173
174     subplot(2,2,4)
175     hold on
176     if length(gamma_range) == 1
177         yline(specific_energy, '-b', 'LineWidth',2)
178         set(gca,'xtick',[])
179         ylim([specific_energy*1.2, 2e6])
180     else
181         plot(gamma_range, specific_energy)
182     end
183     yline(0,'k','BOUND ORBIT LIMIT','LineWidth',2)
184     hold off
185
186     title('Specific energy','FontSize',20)
187 end
188
189 %% ----- CHECKING FOR BOUNDED ORBITS -----
190
191 bound_orbit_index = (specific_energy < 0); % Index of bounded orbits
192 specific_energy = specific_energy(bound_orbit_index); % Specific energy of bounded
orbits
193 a = -mu./(2*specific_energy); % Semi-major axis of bounded orbit
194 final_position = final_position(bound_orbit_index,:); % Position at AE
195 final_velocity = final_velocity(bound_orbit_index,:); % Velocity at AE
196 gamma_bound = gamma_range(bound_orbit_index); % FLight path angle for bounded orbits
197 time_atm_pass = time_atm_pass(bound_orbit_index); % Atmospheric pass time
198 distance_atm_pass = distance_atm_pass(bound_orbit_index);
199
200 %% ----- CALCULATING ECCENTRICITY (e) AND ECCENTRIC ANMALY (E) -----
201
202 [statevector, ~] = MCI20E(final_position', final_velocity'); % Orbital element
determination algorithm
203
204 a = statevector(1,:);
205 e = statevector(2,:);
206 i = statevector(3,:);
207 Omega = statevector(4,:);
208 omega = statevector(5,:);
209 nu = statevector(6,:);
210
211 %% ----- CHECKING APOGEE OF ORBIT IS OUTSIDE MARTIAN ATMOSPHERE -----
212
213 ra = a.*(1+e); % Apogee radii
214 index = ra>(Rm+h_AI);
215
216 specific_energy = specific_energy(index);
217 a = a(index);
218 e = e(index);
219 nu = nu(index);
220 gamma_bound = gamma_bound(index);
221 final_position = final_position(index,:);
222 final_velocity = final_velocity(index,:);
223 time_atm_pass = time_atm_pass(index);
224 distance_atm_pass = distance_atm_pass(index);
225
226 %% ----- OUTPUT -----
227
228 Structure.ballistic.(['BC' num2str(BC)]).(density_label{density_mode}).specific_energy
= specific_energy; % Specific energy of resultant orbit
229 Structure.ballistic.(['BC' num2str(BC)]).(density_label{density_mode}).a = a; % Semi-
```



```
major axis
230     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).nu = nu; % True
    anomaly
231     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).e = e; %
eccentricity
232     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).i = i; %
inclination
233     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).Omega = Omega;
234     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).omega = omega;
235     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).gamma =
gamma_bound'; % Bound orbit gamma
236     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).position =
final_position; % [x , y] coordinates of AE
237     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).velocity =
final_velocity; % [x , y] coordinates of AE
238     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).time_atm_pass =
time_atm_pass;
239     Structure.ballistic(['BC' num2str(BC)]).(density_label{density_mode}).
distance_atm_pass = distance_atm_pass;
240     fprintf('\n')
241
242     toc
243 end
244 if length(BC_range) ~= 1
245     fprintf('\nCOMPLETE: %2.2f %% \n', (BC+1-BC_range(1))/(range(BC_range)+1)*100)
246 end
247
248 %% ----- COMPULSORY PLOT -----
249
250 if length(gamma_range) == 1
251     figure('units','normalized','outerposition',[0 0 1 1])
252
253     Counter=0;
254
255     y(:,1) = rad2deg(y(:,1));
256     y(:,2) = rad2deg(y(:,2));
257     y(:,3) = (y(:,3) - Rm)./1e3;
258     y(:,4) = y(:,4)./1e3;
259     y(:,5) = rad2deg(y(:,5));
260     y(:,6) = rad2deg(y(:,6));
261
262     for i = 1 : size(y,2)
263         if i== 1
264             plot(t,y(:,i),strcat(Markers{mod(Counter,Markermax)+1},Line_format{mod(Counter
,Line_formatmax)+1}),'Color',Color{mod(Counter+1,Colormax)+1})
265             else
266                 addaxis(t,y(:,i),strcat(Markers{mod(Counter,Markermax)+1},Line_format{mod(
Counter,Line_formatmax)+1}),'Color',Color{mod(Counter+1,Colormax)+1})
267             end
268             Counter = Counter+1;
269         end
270
271         ylabel = {'Longitude ($^\circ$)', 'Latitude ($^\circ$)', 'Altitude (km)', 'Velocity (km
/s)', 'FPA ($^\circ$)', 'Heading angle ($^\circ$)'};
272         xlabel('Time (s)')
273
274         hax = getaddaxisdata(gca,'axisdata');
275         hax = [hax{:}];
276         hax = [gca hax(1,:)];
277
278         x0 = 0.1; % spacing between outermost rulers and figure edge
279         xgap = 0.06; % spacing between rulers
280         fontsize = 14; % base font size for all rulers/labels
281
282         % get info about ruler locations
283         nlr = nnz(strcmp({hax.YAxisLocation},'left'));
284         nrr = numel(hax) - nlr;
285         % calculate x-offsets for rulers, reorder to match axes order
286         xposl = x0+(0:(nlr-1)*xgap;
287         xposr = 1-(x0+(nrr-1:-1:0)*xgap);
288         xpos = zeros(1,numel(hax));
289         xpos(1:2:nlr*2) = fliplr(xposl);
290         xpos(2:2:nrr*2) = xposr;
291
292         % width of primary axes
293         wax = 1 - ((nlr-1)+(nrr-1))*xgap - 2*x0;
```



```
294 % apply properties
295 for k = 1:numel(hax)
296     hax(k).Position(1) = xpos(k);
297     hax(k).YLabel.String = ylabel{k};
298     hax(k).FontSize = fontsize;
299 end
300 hax(1).Position(3) = wax;
301
302 leg1 = legend('Longitude','Latitude', 'Altitude','Velocity', 'FPA', 'Heading angle','
Location','southeast','fontsize',18);
303 leg1.TextColor = 'k';
304 leg1.Box = 'off';
305 xlim([0,300])
306 set(leg1,'color','none');
307 set(gca,'box','off','ycolor',Color{2});
308 % set(gca, 'visible', 'off')
309
310 figure('units','normalized','outerposition',[0 0 1 1]);
311 traj_plot_3D([t, R, V, gamma, delta, lambda, chi], true);
312 set(gca, 'FontSize',24)
313
314
315 sgtitle(['Ballistic coefficient  $\beta$  = ',num2str(BC)], 'fontsize',20)
316
317 subplot(1,2,1)
318
319 yyaxis left
320 plot(t,h/1e3,'b')
321 ylabel('Altitude (Km)')
322 yline(0,'-.b','SURFACE','LineWidth',2)
323 yline(h_AI/1e3,'-.b','ATMOSPHERIC LIMIT','linewidth',2 )
324 ylim([-10, h_AI/1e3+20])
325
326 yyaxis right
327 plot(t,V/1e3,'r')
328 ylabel('Velocity (Km/s)')
329 ylim([0,inf])
330
331 title([' $\gamma_0$  = ', num2str((gamma0))], 'FontSize',20)
332 xlabel('Time (s)')
333 xlim([0,t(end)])
334
335 legend({'Altitude','','', 'Velocity'},'Location','east','FontSize',18)
336 subplot(1,2,2)
337 traj_plot_3D([t, R, V, gamma, delta, lambda, chi], plot_option);
338
339 end
340
341 fprintf('\n')
342
343 end
```

G.2 MATLAB Code: Re-entry dynamics

```
1 function [y] = reentry_dynamics(t, x, BC, density_mode)
2
3 %% ----- GLOBAL PARAMETERS -----
4
5 global Rm m mu L_D
6
7 Omega = 2*pi / (24*3600 + 38*60 + (22+35)/2);
8
9 delta = x(1); % Longitude
10 lambda = x(2); % Latitude
11 R = x(3); % Position
12 V = x(4); % Velocity
13 gamma = x(5); % FPA
14 chi = x(6); % HEading angle (pi/2 - A)
15
16 w3 = 0;
17
18 g = mu/R^2; % Gravitational accelertaion
19
20
21 % Atmopsheric parameters
22
```



```
23 if R < Rm
24     R = Rm;
25 elseif R > Rm + 200e3
26     R = Rm + 200e3;
27 end
28
29 h = R - Rm; % Altitude
30
31 [~, ~, rho, ~] = marsatm(h, density_mode, 0, 0); % Atmospheric properties
32 q_inf = 0.5*rho*V^2; % Dynamic head
33
34 %% ----- AERODYNAMIC MODEL -----
35 % DRAG MODEL
36 D = q_inf .* m./BC *1.35;
37
38 % LIFT MODEL
39 L = D.*L_D;
40
41 % FORCES
42 Xfo = L;
43 Yfo = -D;
44 Zfo = 0;
45
46 %% ----- GOVERNING EQUATIONS -----
47
48 y(1) = V*cos(gamma)*cos(chi)/(R*cos(lambda));
49 y(2) = V*cos(gamma)*sin(chi)/(R);
50 y(3) = V*sin(gamma);
51 y(4) = -g*sin(gamma) + Yfo/m; + Omega^(2)*R*cos(lambda)*...
52     (sin(gamma)*cos(lambda) - cos(gamma)*sin(chi)*sin(lambda));
53 y(5) = (V/R-g/V)*cos(gamma) + Xfo/(V*m) + 2*Omega*cos(chi)*cos(lambda)+ ...
54     (Omega^2*R*cos(lambda)*(cos(gamma)*cos(lambda)+sin(gamma)*sin(chi)*sin(lambda)))/V;
55 y(6) = -V*cos(gamma)*cos(chi)*tan(lambda)/R + Zfo/(m*V*cos(gamma)) + ...
56     2*Omega*(tan(gamma)*sin(chi)*cos(lambda)-sin(lambda))-(Omega^2*w3*cos(chi)*cos(lambda)*sin
57     (lambda))/(V*cos(gamma));
58
59 y = y';
```

H MATLAB Code: AsTRO Algorithm

```
1
2 Markers = {'.', '+', 'x', '*', 'o', 'v', 'd', '^', 's', '>', '<'};
3 Line_format={'-', '--', ':'};
4 Color={'b', 'k', 'r', 'm', 'g', [255, 165, 0]/255};
5 Counter = 0;
6 Markermax=length(Markers);
7 Line_formatmax=length(Line_format);
8 Colormax=length(Color);
9
10 global Rm mu h_AI m L_D L T_w_start models vinf CD
11
12 %% LOADING DATA
13
14 load('Scenario1.mat')
15 load('Scenario2.mat')
16
17 % ADEQUATE
18 load('MarsGRAMdensity.mat')
19 load('MarsGRAMdensity_hi3sd.mat')
20 load('MarsGRAMdensity_lo3sd.mat')
21
22 models.MarsGRAMdensity = MarsGRAMdensity;
23 models.MarsGRAMdensity_hi = MarsGRAMdensity_hi3sd;
24 models.MarsGRAMdensity_lo = MarsGRAMdensity_lo3sd;
25 %% CONSTANTS
26
27 Rm = 3389.5e3;
28 mu = 4.282837e13;
29 m = S1.mass;
30 L_D = 0.2;
31 L = 2.6;
32 T_w_start = 290;
33 CD = 1.55;
34
35 vinf = 3.5;
36
```




```
37 V_hype_fun = @(r) 2*sqrt((vinf/1e3).^2./2 + mu*10^-9./(r/1e3)) ;
38 h_AI = 125e3;
39
40 % Atmospheric interface
41 r_AI = Rm+h_AI;
42 a_hyper = -mu/(vinf^2);
43 specific_energy_hyper = vinf^2/2;
44 v_AI = V_hype_fun(r_AI)*1e3;
45
46 %%
47
48 load('Lifted_corridor_2_states6_final_insertion_actual_model.mat')
49 load('Scenario1.mat')
50
51 S = Lifted_corridor_2_states6;
52
53 BC = S.BC;
54 delta_gamma = S.delta_gamma;
55 lb_gamma = max(S.lower_bound,[],1);
56 ub_gamma = min(S.upper_bound,[],1);
57 mid_gamma = (ub_gamma-S.delta_gamma/2);
58
59 %% SIMULATE
60 density_mode = 3;
61
62 % Variable pre-allocation
63 a = zeros(1,length(BC));
64 e = zeros(1,length(BC));
65 inclination = zeros(1,length(BC));
66 Omega = zeros(1,length(BC));
67 omega = zeros(1,length(BC));
68 nu = zeros(1,length(BC));
69 q_dot_max = zeros(1,length(BC));
70 Q = zeros(1,length(BC));
71 T_wall = zeros(1, length(BC));
72
73
74 delta0 = deg2rad(0.5798); % Initial Longitude, +ve to east
75 lambda0 = deg2rad(34.49); % Initial Latitude, +ve northern hemisphere
76 R0 = Rm + h_AI; % Atmospheric interface distance from martian centre
77 V0 = v_AI; % Atmospheric interface velocity
78 chi0 = deg2rad(90-108.24); % heading angle
79 % chi0 = deg2rad(90-160); % heading angle
80 gamma0 = mid_gamma;
81
82 % ODE parameters
83 tspan = [0, 700]; % Time span being analysed
84 Opt = odeset('Events', @myEventLimit, 'RelTol',1e-4,'AbsTol',1e-4); % Solver option which
    % terminates run when R state reached atmospheric boundary or martian surface
85 % Opt = odeset('Events', @myEventLimit);
86
87 for i = BC
88     % Setting up ODE solver
89     x0 = [delta0, lambda0, R0, V0, deg2rad(gamma0(i+1-BC(1))), chi0];
90     [t,y] = ode23s(@(t,x) reentry_model2(t, x, i, density_mode), tspan, x0, Opt);
91
92     lat = y(:,1);
93     lon = y(:,2);
94     R = y(:,3);
95     h = R - Rm;
96     V = y(:,4);
97     gamma = y(:,5);
98     chi = y(:,6);
99
100     % Thermal model
101     [q_dot, q_dot_max(i+1-BC(1)), Q_dist, T_wall(i+1-BC(1))] = thermalModelSegu(t, R - Rm, V,
102     1, density_mode);
103     Q(i+1-BC(1)) = Q_dist(end);
104
105     % Orbital params
106     specific_energy = 0.5*V(end)^2 - mu/R(end);
107     [final_position, final_velocity] = traj_plot_3D([t, R, V, gamma, lat, lon, chi] , false);
108     [statevector, ~] = MCI2OE(final_position, final_velocity);
109     a(i+1-BC(1)) = statevector(1,:);
110     e(i+1-BC(1)) = statevector(2,:);
111     inclination(i+1-BC(1)) = statevector(3,:);
```



```
111     Omega(i+1-BC(1)) = statevector(4,:);
112     omega(i+1-BC(1)) = statevector(5,:);
113     nu(i+1-BC(1)) = statevector(6,:);
114 end
115
116 params.sim = [a; e; inclination; q_dot_max; Q; T_wall];
117 params.operational = [S1.a.max; 70];
118
119 %optimization_target = [volume umbrella, Delta V, Q, q dot max, T wall, T orbital]
120 T_orbi_target = 2*pi * sqrt(S1.a.max^3/mu);
121 opti_target = [0, 0, 0, 0, 0, 2*pi*sqrt((1000e3+Rm)^3/mu)/3600];
122 weight = [1, 3, 1, 3, 1, 3];
123 [cost, x] = cost_function(mid_gamma, BC, params, opti_target, weight);
124
125 x(2,:) = x(2,+)/1e3; % Convert Delta V from m/s to Km/s
126 x(3,:) = x(3,+)/1e6; % Convert heat load from J/m2 to MJ/m2
127 x(4,:) = x(4,+)/1e4; % Convert heat flux from W/m2 to W/cm2
128 x(6,:) = x(6,+)/3600; % Convert orbital period from seconds to Hrs
129
130 index_minima = [find(cost(1,:) == min(cost(1,:))), find(cost(2,:) == min(cost(2,:)))] ;
131 BC_opt = BC(index_minima);
132 gamma_opt = mid_gamma(index_minima);
133
134 %%
135
136 if index_minima(1) == index_minima(2)
137     index_minima = index_minima(1);
138 end
139
140 var_names = {'Cost-function', 'Ballistic coefficient $(\beta)$', 'Flight path angle $(\gamma)$'};
141 cost_names = {'Least-Square method', 'Pseudo-Huber loss function'};
142 T1 = table(cost_names, BC_opt, gamma_opt, 'VariableNames', var_names);
143 disp(T1)
144
145 an = axial_acceleration([R, V, gamma, lon, chi], BC);
146
147 var_names = {'Max heat flux', 'Heat load', 'Max axial acceleration'};
148 T1 = table(q_dot_max(index_minima), Q(index_minima), max(an(:, index_minima)), 'VariableNames',
149     var_names);
149 disp(T1)
150
151
152
153 %% PLOT
154
155 figure('units','normalized','outerposition',[0 0 1 1]);
156 hold on
157 for i = 1 : size(x,1)
158     if i== 1
159         plot(BC,x(i,:),strcat(Markers{mod(Counter,Markermax)+1},Line_format{mod(Counter,
160             Line_formatmax)+1}), 'Color',Color{mod(Counter+1,Colormax)+1})
161     else
162         addaxis(BC,x(i,:),strcat(Markers{mod(Counter,Markermax)+1},Line_format{mod(Counter,
163             Line_formatmax)+1}), 'Color',Color{mod(Counter+1,Colormax)+1})
164     end
165     Counter = Counter+1;
166 end
167 xline(BC_opt,'-',[ 'Optimal \beta = ', num2str(BC_opt)], 'LineWidth',2, 'FontSize',18)
168 hold off
169
170 ylabel = {'Volume ($m^{-3}$)', '$\Delta V$ (Km/s)', 'Heat load (MJ/m2)', 'Max. heat flux (W/
171     cm2)', 'Wall temp. (K)', 'Orbital period (hrs)'};
172 xlabel('Ballistic coefficient $(\beta)$')
173
174 hax = getaddaxisdata(gca,'axisdata');
175 hax = [hax{:}];
176 hax = [gca hax(1,:)];
177
178 x0 = 0.1; % spacing between outermost rulers and figure edge
179 xgap = 0.06; % spacing between rulers
180 fontsize = 14; % base font size for all rulers/labels
181
182 % get info about ruler locations
183 nlr = nnz(strcmp({hax.YAxisLocation}, 'left'));
184 nrr = numel(hax) - nlr;
```



```
182 % calculate x-offsets for rulers, reorder to match axes order
183 xposl = x0+(0:nlr-1)*xgap;
184 xposr = 1-(x0+(nrr-1:-1:0)*xgap);
185 xpos = zeros(1,numel(hax));
186 xpos(1:2:nlr*2) = fliplr(xposl);
187 xpos(2:2:nrr*2) = xposr;
188
189 % width of primary axes
190 wax = 1 - ((nlr-1)+(nrr-1))*xgap - 2*x0;
191 % apply properties
192 for k = 1:numel(hax)
193     hax(k).Position(1) = xpos(k);
194     hax(k).YLabel.String = ylabel{k};
195     hax(k).FontSize = fontsize;
196 end
197 hax(1).Position(3) = wax;
198
199 leg1 = legend('', 'Volume', '\Delta V', 'Heat load', 'Max. heat flux', 'Wall temp.', 'Orbital
    period');
200 leg1.TextColor = 'k';
201 leg1.Box = 'off';
202
203 set(leg1, 'color', 'none');
204 set(leg1, 'FontSize', 24);
205 set(gca, 'box', 'off', 'ycolor', Color{2});
206
207 %%
208 figure
209
210 yyaxis left
211 plot(BC, cost(1,:))
212 ylabel('Least-square method')
213 xline(BC_opt(1), '-', ['Optimal \beta = ', num2str(BC_opt(1))], 'LineWidth', 2, 'FontSize', 18, '
    LabelHorizontalAlignment', 'left', 'Color', [0, 0.4470, 0.7410])
214 % ylim([-60, 300])
215
216 yyaxis right
217 plot(BC, cost(2,:))
218 ylabel('Pseudo-Huber lost function')
219 xline(BC_opt(2), '-', ['Optimal \beta = ', num2str(BC_opt(2))], 'LineWidth', 2, 'FontSize', 18, '
    Color', [0.8500, 0.3250, 0.0980])
220 % ylim([-0.02, 0.1])
221
222 xlabel('Ballistic coefficient $(\beta)$')
223 title('Cost function comparison')
224
225 grid minor
226 set(gca, 'GridAlpha', 0.4, 'MinorGridAlpha', 0.7);
227 set(gca, 'FontSize', 18)
228 %%
229
230 load('trajectory_states.mat')
231
232 figure
233 plot(array(:,1), array(:,8))
```

H.1 MATLAB Code: Cost Function

```
1
2 function [cost, x] = cost_function(gammas, BC, params, opti_target, weight)
3
4
5 global mu CD m
6
7 %% VARIABLE DESIGNATION
8 sim_params = params.sim;
9 operational_orbit = params.operational;
10
11 a1 = sim_params(1,:); % Semi-major axis of post aerocapture orbit
12 e1 = sim_params(2,:); % Eccentricity of post aerocapture orbit
13 i1 = sim_params(3,:); % Inclination of post aerocapture orbit
14 q_dot_max = sim_params(4,:); % Maximum heat flux of aerocapture trajectory
15 Q = sim_params(5,:); % Heat load of aerocapture trajectory
16 Tw = sim_params(6,:); % Rough wall temperature
17 ra1 = a1.*(1+e1); % Apogee radius of post aerocapture orbit
18
```



```
19 a2 = operational_orbit(1,:); % Semi-major axis of operational orbit
20 i2 = operational_orbit(2,:); % Inclination of operational orbit
21
22 %% VARIABLE PRE-ALLOCATION FOR SPEED
23 x = zeros(6,length(gammas));
24
25 %% VOLUME AND MASS
26 V_umbrella = pi*1.3^2.*(sqrt(m./BC.*CD*pi)-1.3);
27
28 %% DELTA V COST FUNCTION
29 theta = deg2rad(i2) - i1; % Orbital inclination change
30 theta(theta<=0) = 0; % If orbital inclination is already 70 or more, no burn is required
31 at = 0.5.*(ra1+a2); % Semi-major axis of transfer orbit
32 via = sqrt(mu.*(2./ra1-1./a1)); % Velocity at apogee of post aerocapture orbit
33
34 % Hoffman transfer Delta V
35 DeltaV_hoffman = abs(sqrt(mu) .* (sqrt(1./a2)+sqrt(2./ra1-1./at)-sqrt(2./a2-1./at)-sqrt(2./ra1
    -1./a1))); % Delta V for hoffman transfer
36 % DeltaV_hoffman = 0;
37
38 % Plane change Delta V
39 DeltaV_plane = abs(2*via.*sin(theta/2));
40
41 % Total Deltav V
42 DV_total = (DeltaV_plane + DeltaV_hoffman)';
43
44 %% ORIBTAL PERIOD
45 T0 = 2*pi .*sqrt(a1.^3./mu);
46
47 x(1,:) = V_umbrella;
48 x(2,:) = DV_total;
49 x(3,:) = Q;
50 x(4,:) = q_dot_max;
51 x(5,:) = Tw;
52 x(6,:) = T0;
53
54
55 %% FEATURE SCALING AND MEAN NORMALIZATION
56 x_mean = mean(x,2);
57 sigma = std(x,[],2);
58
59 x_norm = (x-x_mean)./(sigma);
60 x_norm(isnan(x_norm)) = 0;
61 %% COST FUNCTION
62
63 cost(1,:) = least_square_method(x_norm, opti_target, weight);
64 cost(2,:) = pseudo_huber_loss(x_norm, opti_target, 0.1, weight) ;
65 end
```